Kaip sujungti nesuderinamas sistemas, kad jos bendradarbiautų?

Kaip sustabdyti klasių sprogimą ir atskirti tai, kas keičiasi?

Kaip paslėpti sistemos sudėtingumą už vienos paprastos sąsajos?

T120B516 Objektinis programų projektavimas

2026 rudens semestras

Dr. Andrej Ušaniov

Jus esate čia

Kodėl tai svarbu? Nes kitą savaitę kontrolinis ☺

T120B516 Paskaitų planas 2026 m. rudens semestras

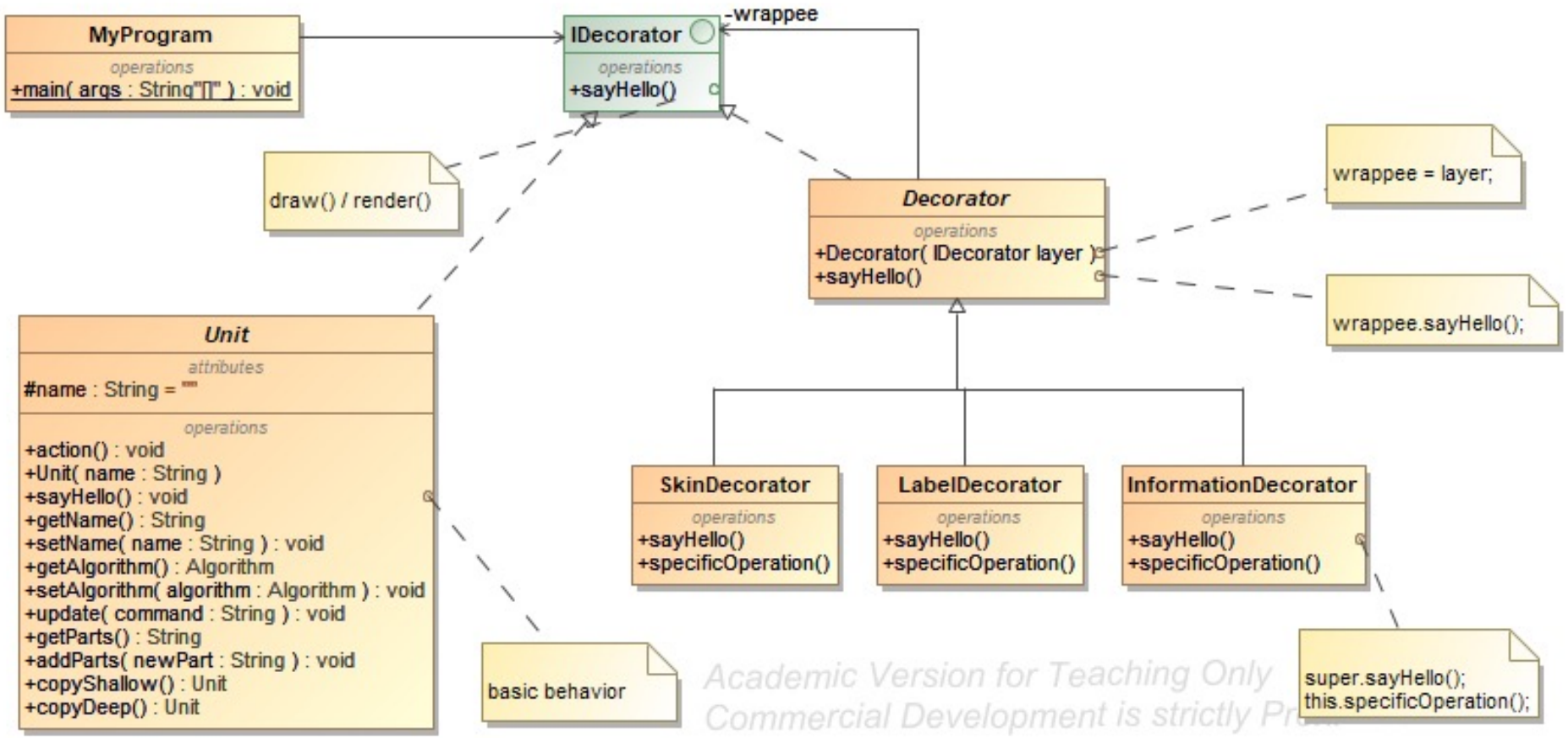
Sav. Nr.	Paskaitos data	Paskaitų temos	Projektinio darbo veiklos	Projektinės veiklos ir testo periodas	Testai
1			Sudaryti grupę, Suderint temą	iki 2026-09-08	
2	2026-09-07	Apie modulį, Įžanga į projektavimo šablonus, Hello World	Pristatyti projekto aprašymą, UML Use case, Class diagram, Activity diagram, Reikalavimai	iki 2026-09-15	
3	2026-09-14	*Projektavimo šablonų apžvalga, Signal-R apžvalga, UML ir OOP principai*	Pristatyti žaidimo prototipą (tinklo komunikacija, grafika, valdymas), Klasių diagrama	iki 2026-09-22	
4	2026-09-21	01-Singleton, 02-Factory, 03-Abstract Factory	Šablonų taikymas: 01-Singleton, 02-Factory, 03-Abstract Factory, 04-Strategy, 05-Observer, 06-Builder, 07-Prototype	iki 2026-09-29	
5	2026-09-28	04-Strategy, 05-Observer		iki 2026-10-06	
6	2026-10-05	06-Builder, 07-Prototype		iki 2026-10-13	
7	2026-10-12	08-Decorator, 09-Command	Šablonų taikymas: 08-Decorator, 09-Command, 10-Adapter, 11-Façade, 12-Bridge	iki 2026-10-20	
8	2026-10-19	10-Adapter, 11-Façade, 12-Bridge, Pasiruošimas K1		iki 2026-10-27	
9	2026-10-26	K1 per Moodle ir MS Teams su įjungta kamera	**P1 gynimas**	iki 2026-11-03	
10	2026-11-02	Vėlinės - paskaitos nebus	**P1 pakartotinas gynimas**	iki 2026-11-10	**T1**
11	2026-11-09	13-Template Method, 14-Iterator,	Šablonų taikymas: 13-Template method, 14-Iterator, 15-Composite, 16-Flyweight, 17-State, 18-Proxy, 19-ChainOfRespnsibility	iki 2026-11-17	
12	2026-11-16	15-Composite, 16-Flyweight		iki 2026-11-24	
13	2026-11-23	17-State, 18-Proxy, 19-ChainOfResponsibility		iki 2026-12-01	
14	2026-11-30	20-Interpreter, 21-Mediator	Šablonų taikymas: 20-Interpreter, 21-Mediator, 22-Memento, 23-Visitor	iki 2026-12-08	
15	2026-12-07	22-Memento, 23-Visitor		iki 2026-12-15	**T2**
16	2026-12-14	Pasiruošimas egzaminui, Projektinių darbų pristatymas	**P2 gynimas**	iki 2026-12-22	
17	2026-12-21	K1 perlaikymas auditorijoje	**P2 pakartotinas gynimas**	iki 2026-12-22 imtinai	
18-21	2027-01-??	Egzaminas			

Ką prisimenate iš praėjusio karto?

- **Decorator** - keičia tai, KAS vyksta aplink veiksmą

- **Command** - paverčia patį VEIKSMĄ objektu

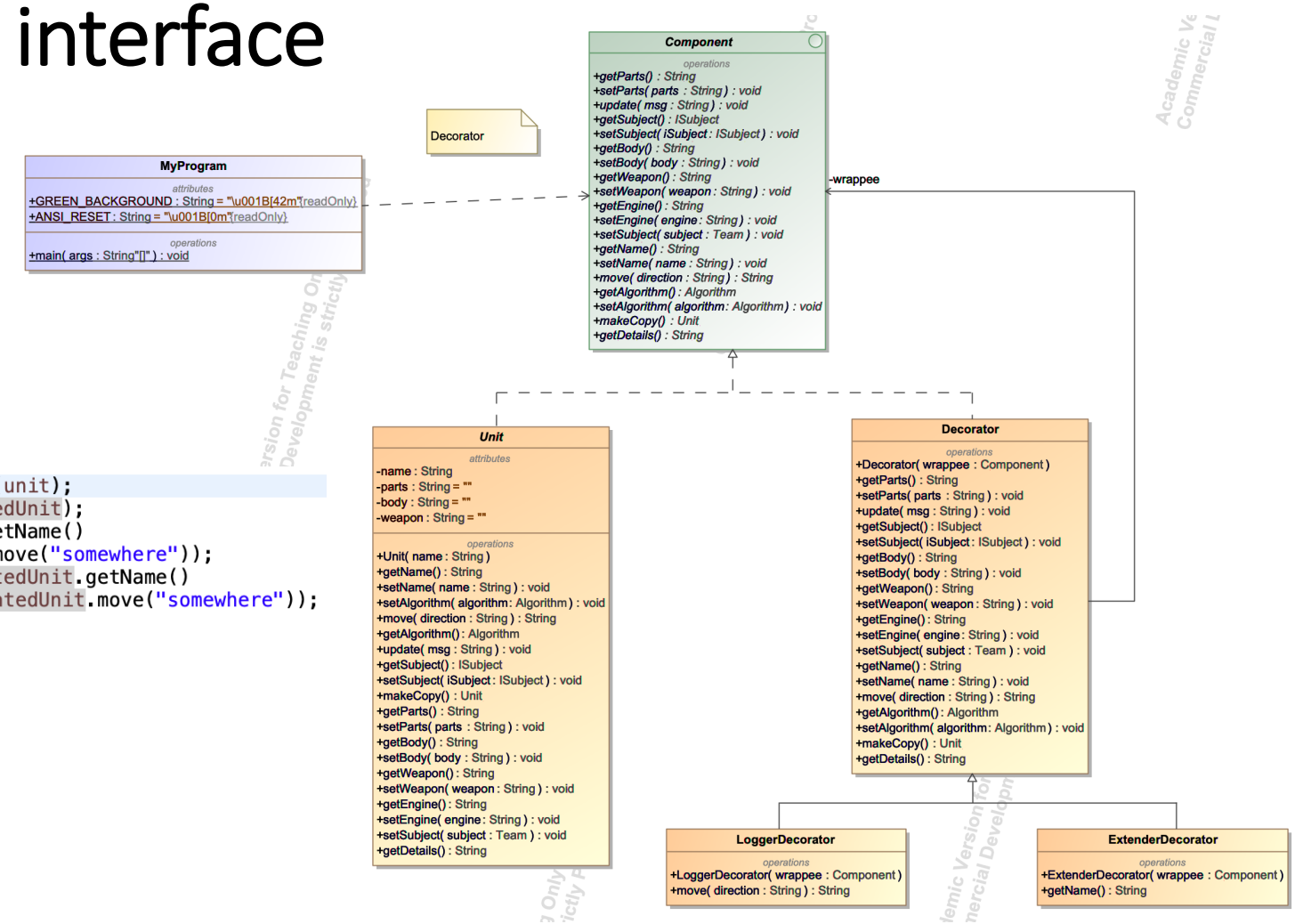
Decorator Narrow Interface



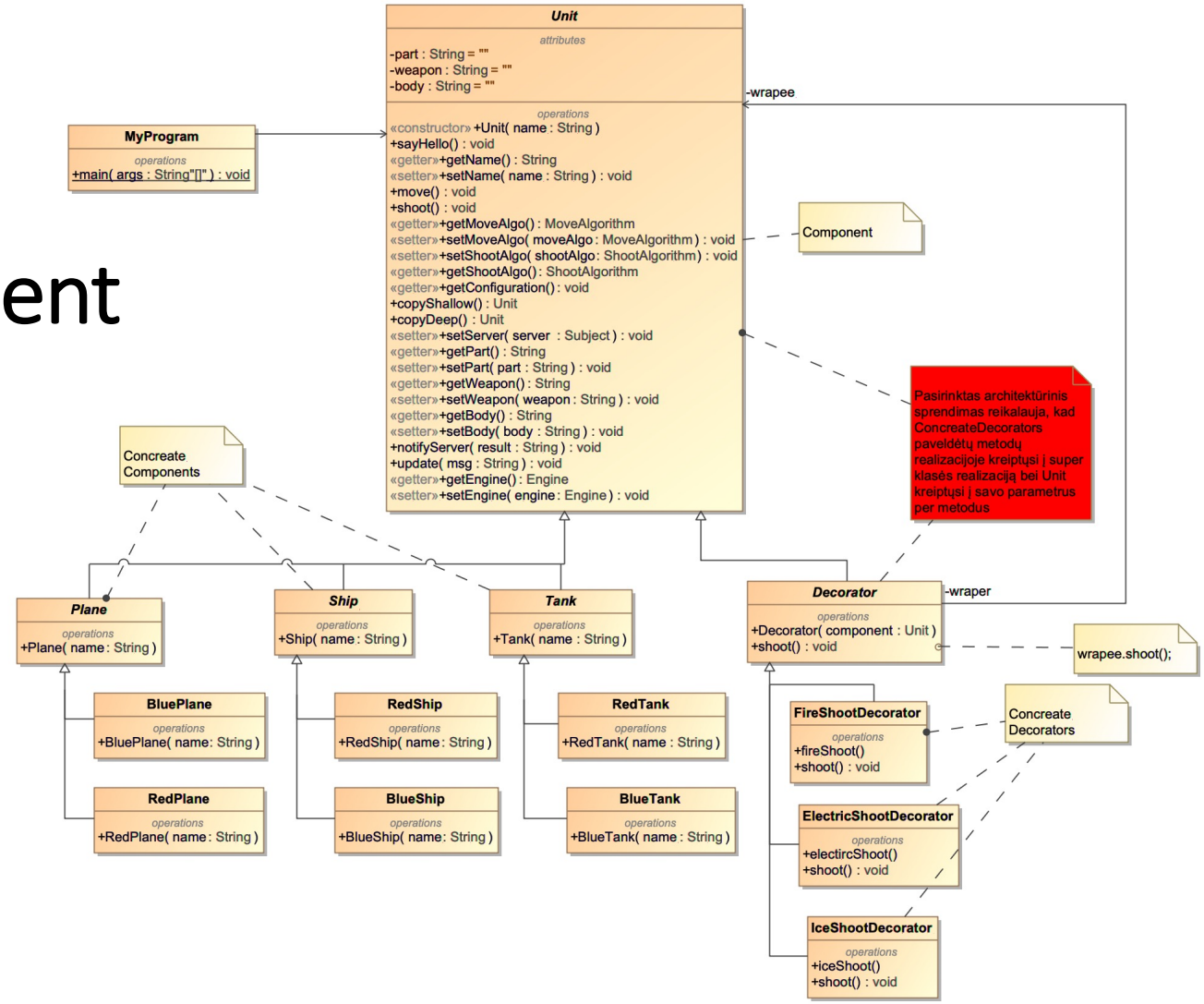
Academic Version for Teaching Only
Commercial Development is strictly Pro

Decorator Wide interface

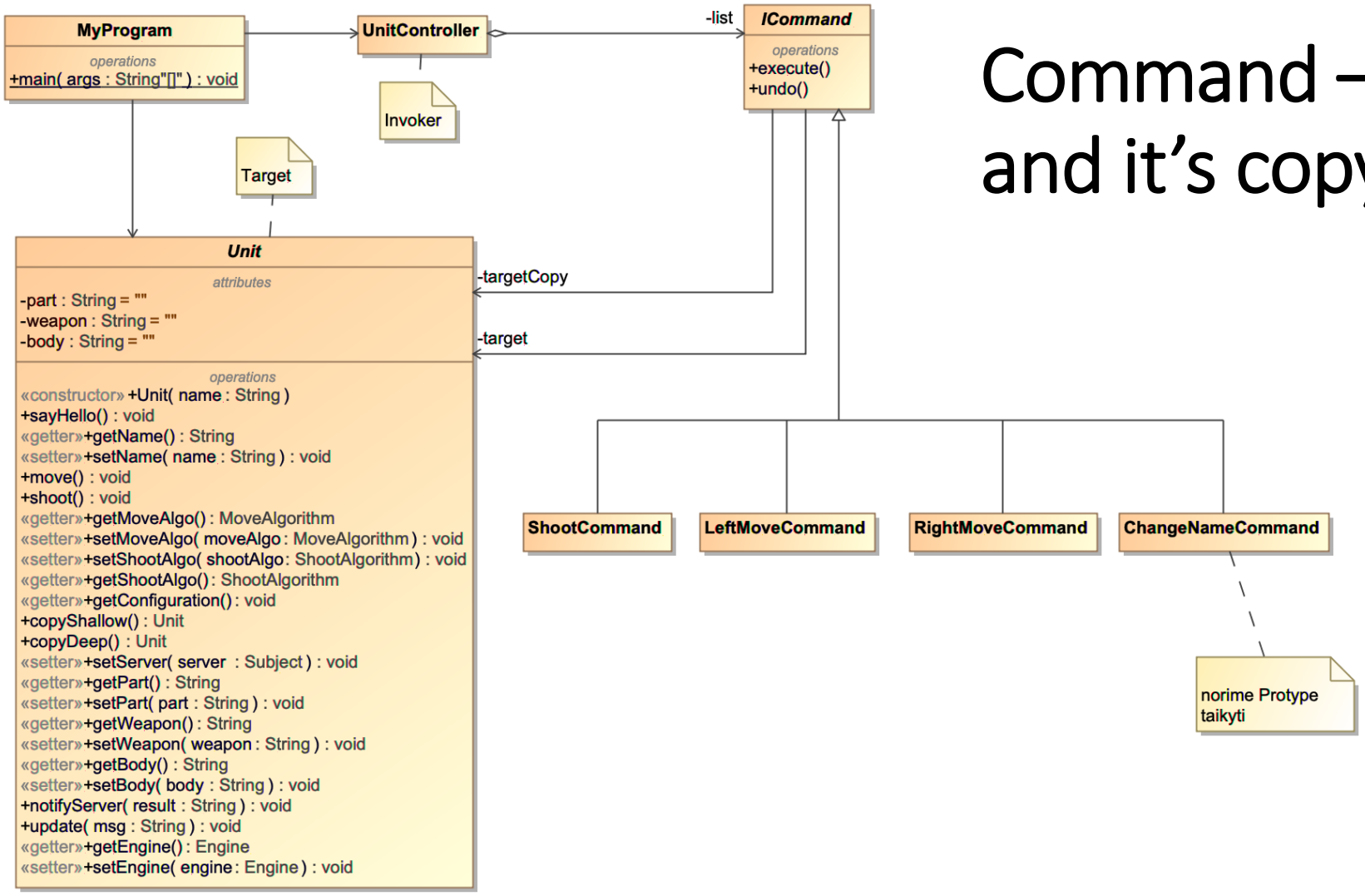
```
Component decoratedUnit = new LoggerDecorator(unit);
decoratedUnit = new ExtenderDecorator(decoratedUnit);
System.out.println("    " + "Unit:  " + unit.getName()
                   + ", move: " + unit.move("somewhere"));
System.out.println("    " + "Unit:  " + decoratedUnit.getName()
                   + ", move: " + decoratedUnit.move("somewhere"));
```



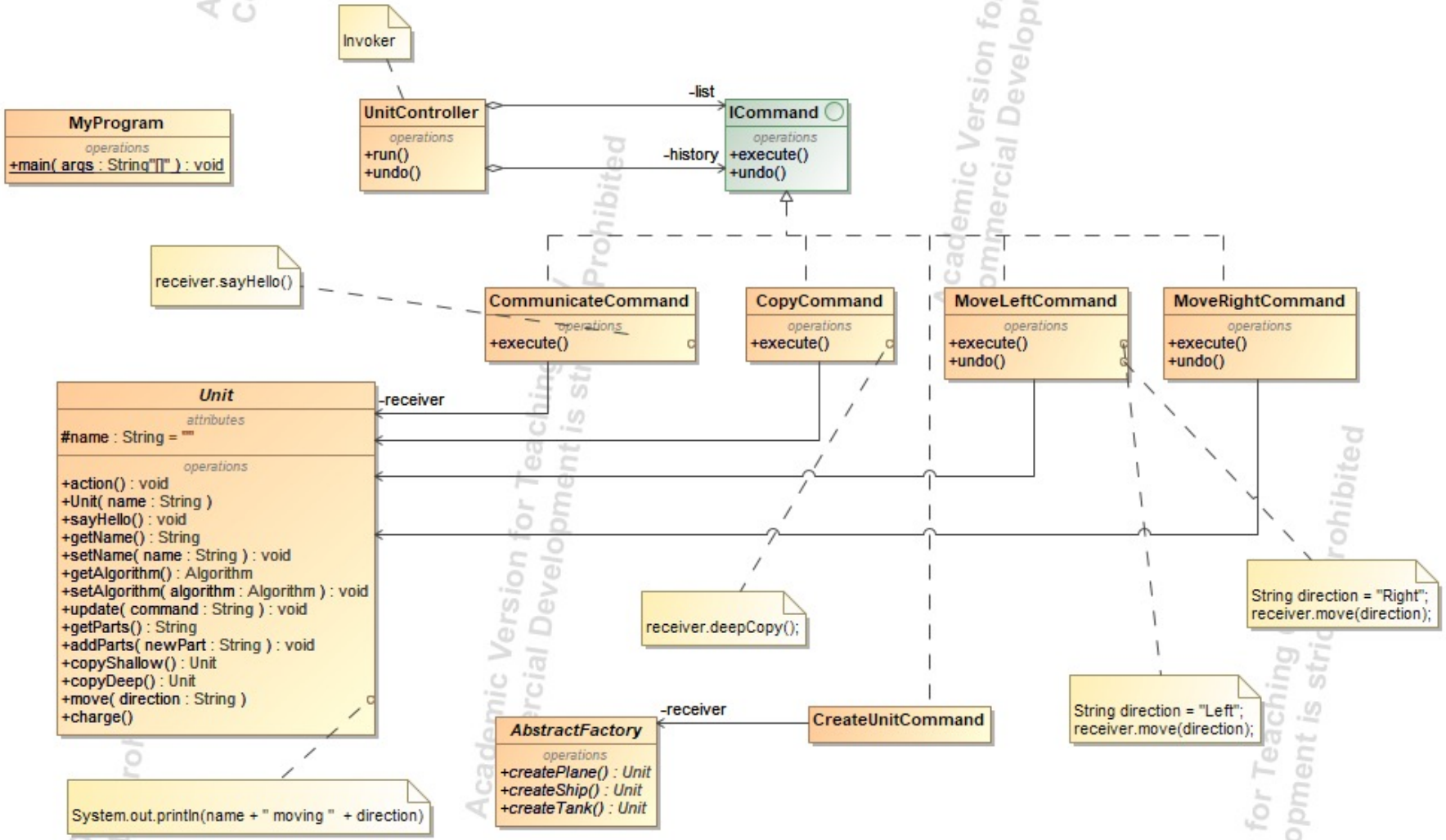
Decorator – ConcreteComponent as a Component



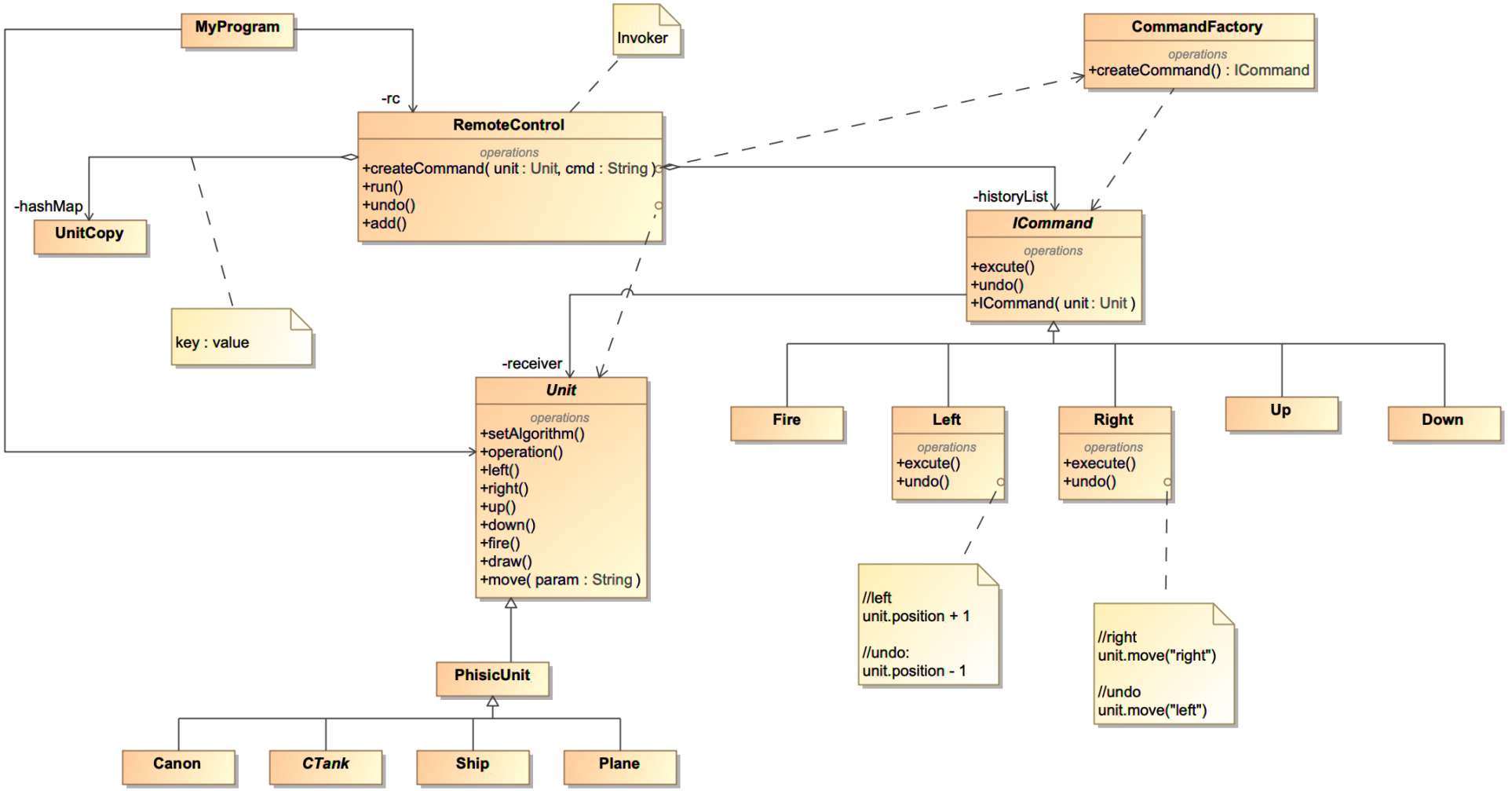
Command –target and it's copy



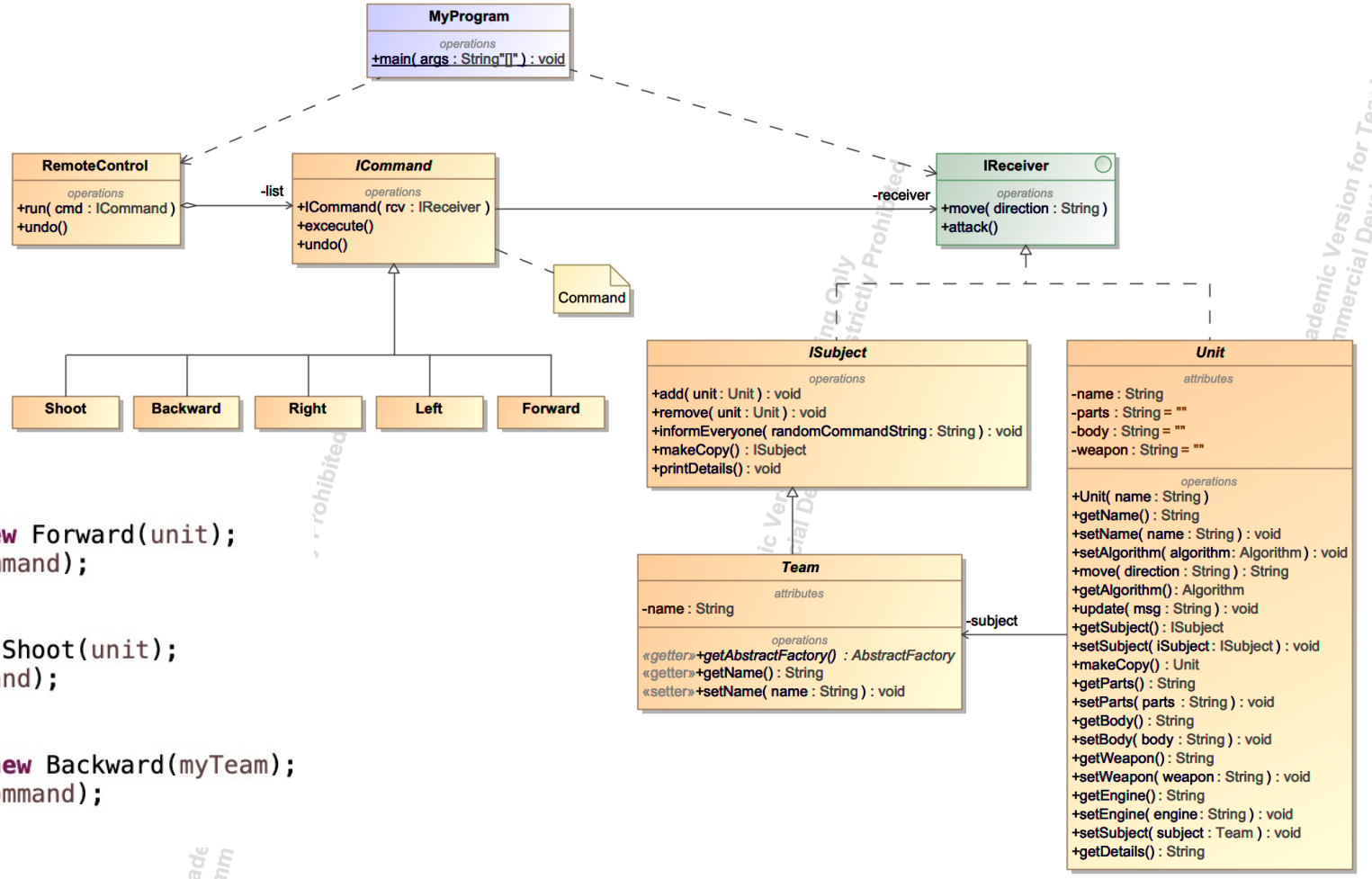
Command – different receivers



Command with CommandFactory



Command - Team and Unit as Observer and Subject

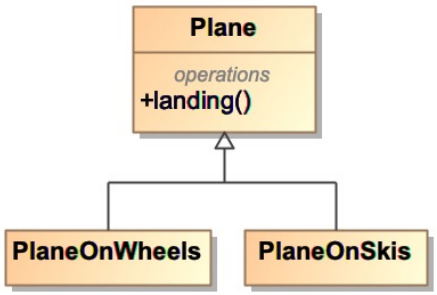


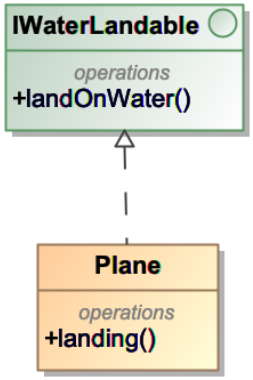

```
ICommand forwardCommand = new Forward(unit);
remoteControl.run(forwardCommand);
remoteControl.undo();

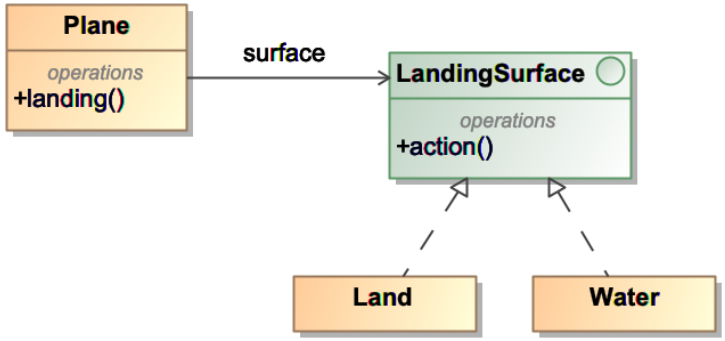
ICommand shootCommand = new Shoot(unit);
remoteControl.run(shootCommand);
remoteControl.undo();

ICommand backwardCommand = new Backward(myTeam);
remoteControl.run(backwardCommand);
remoteControl.undo();
```

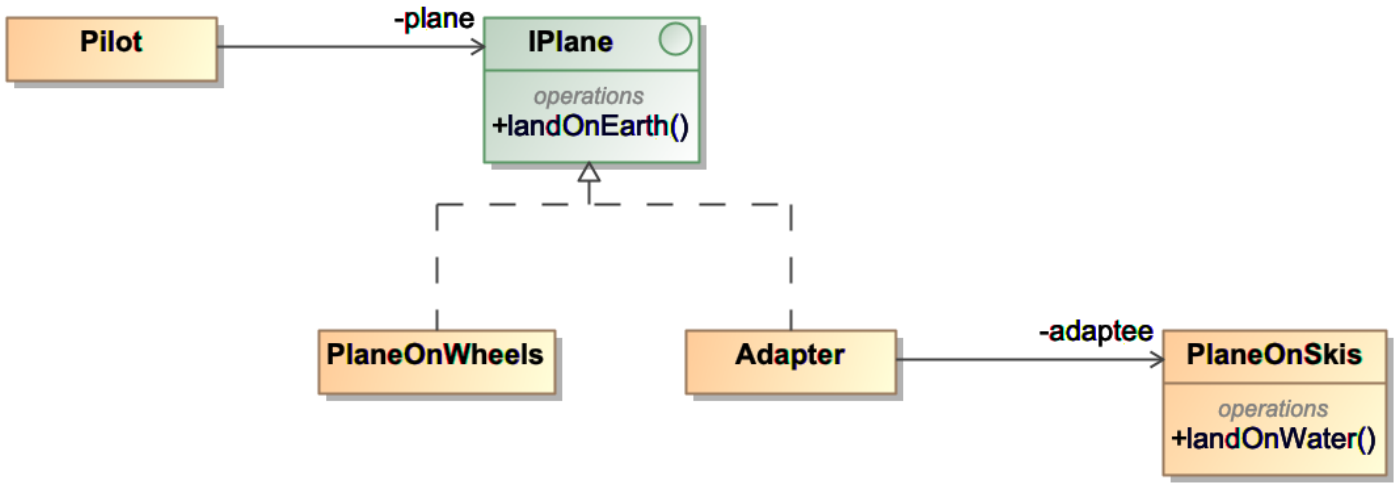
Turime lėktuvą, norime, kad pilotas mokėtų jį nutupdyti ant vandens







O ką jei panaudosime adapterį?



- Adapter nekuria naujo funkcionalumo. Adapter tik leidžia bendradarbiauti klasėms, kurios kitaip nesusišnekėtų.

Adapter – prisitaiko prie nesuderinamos sąsajos

- Type: Structural
- Also Known As: Wrapper
- Intent: “Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces”

When to use “Adapter”, motivation

- When existing class and its **interface doesn't match the domain-specific interface** an application requires
- When you want to **create a reusable class** that **cooperates** with unrelated or **unforeseen classes**, that don't necessarily have compatible interfaces
- if rewriting legacy code isn’t an option

- Drawbacks of adapters: **additional layer of code** added, and so to maintain

Adapter examples.





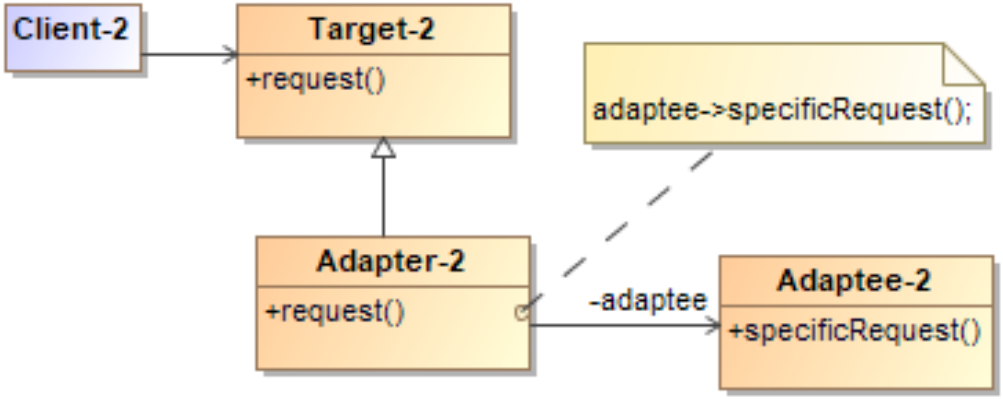




Adapter's structure (GoF)

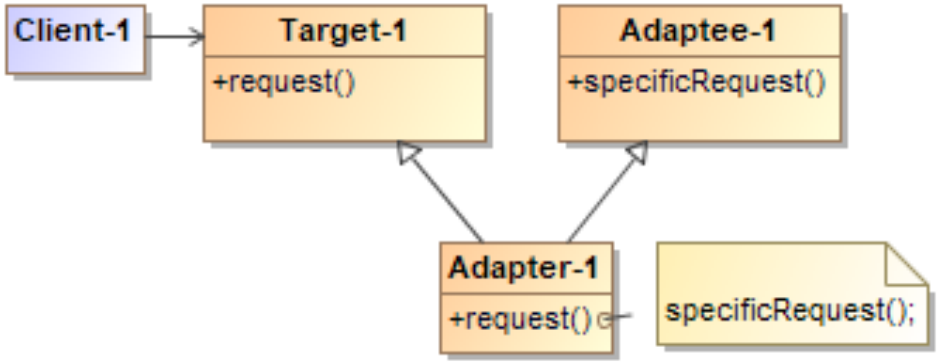
An object adapter

relies on object composition



A class adapter (two-ways adapter)

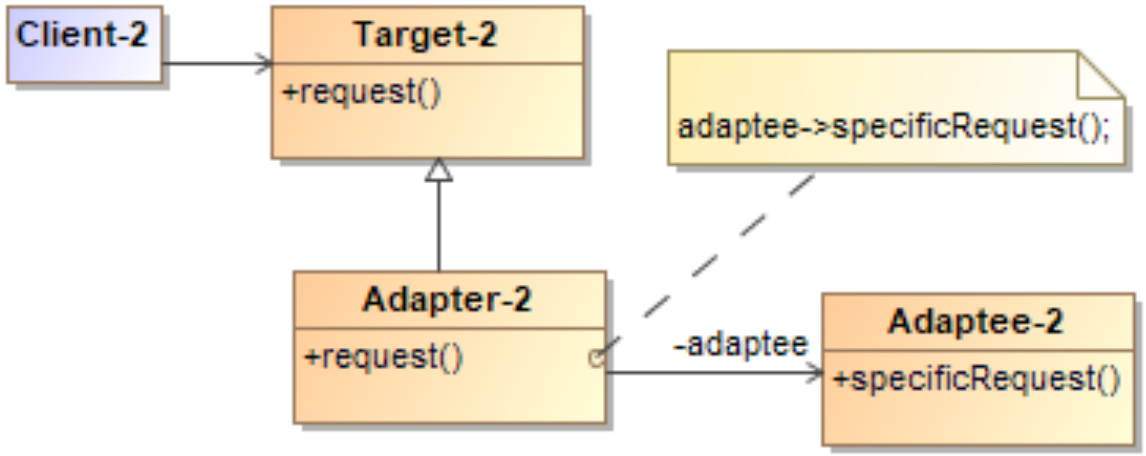
uses multiple inheritance to adapt one interface to another



A potential problem of object adapter: adapters aren't transparent to all clients. An adapted object no longer conforms to the Adaptee interface, so it can't be used as is wherever an Adaptee object can.

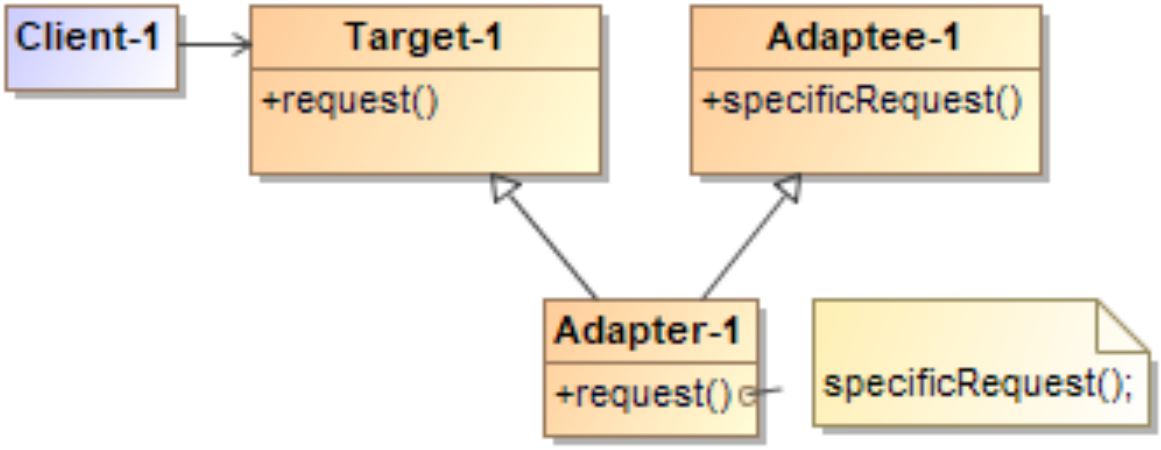
Two-way adapters can provide such transparency. Specifically, they're useful when two different clients need to view an object differently.

An Object Adapter



- Allows a single Adapter work with many Adaptees (the Adaptee itself and its subclasses).
- Can add functionality to all Adaptees at once
- Harder to override Adaptee behavior (subclassing Adaptee and referring Adapter to the subclass rather than the Adaptee itself)

A class adapter (Two-ways adapter)



- Won't work to adapt a subclasses of “Adaptee-1”class
- Allows Adapter override some of Adaptee's behavior
- Java, C# do not support multiple class inheritance (workaround Interfaces)

Adapter Pros and Cons

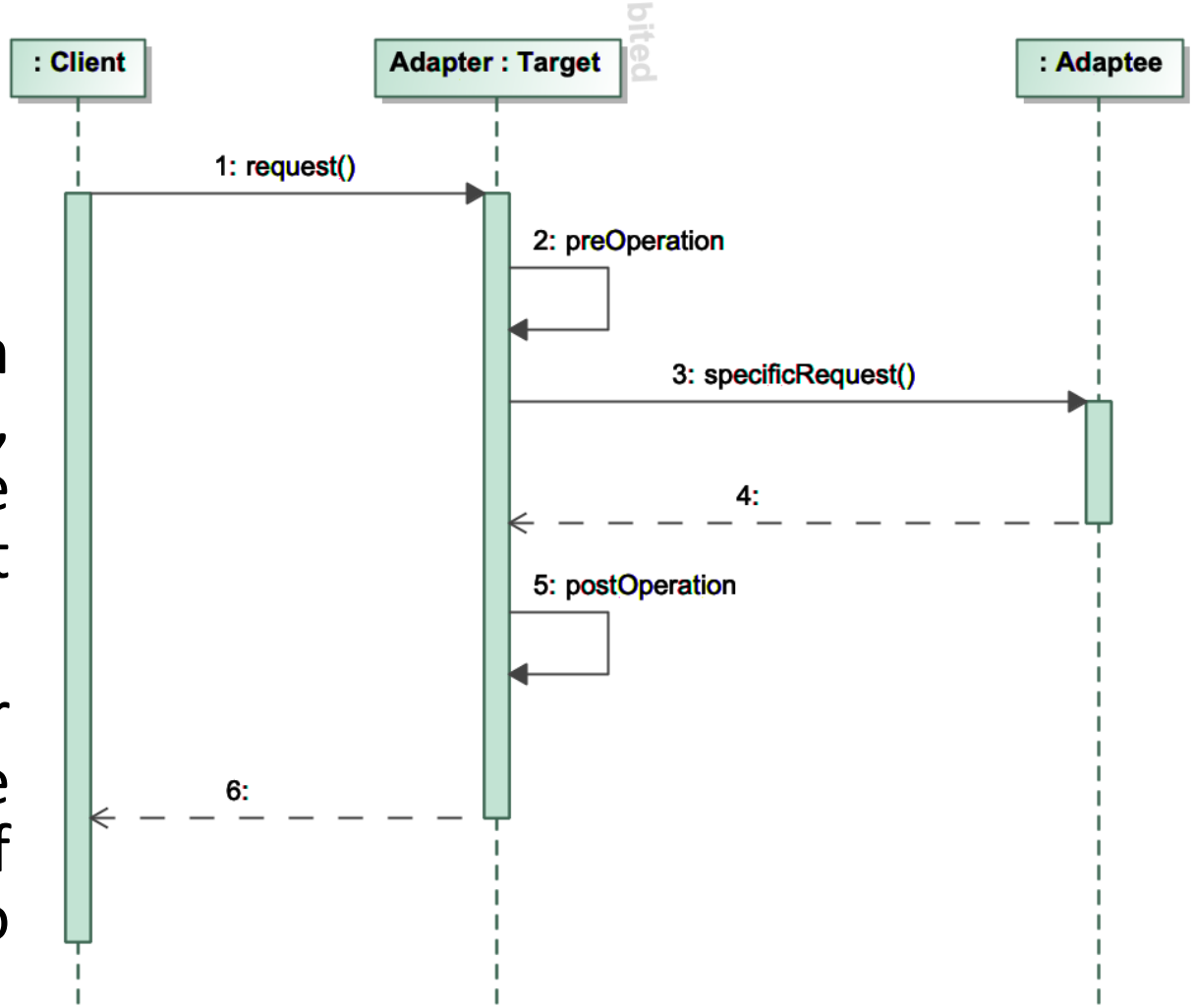
Single Responsibility Principle. You can separate the interface or data conversion code from the primary business logic of the program.

Open/Closed Principle. You can introduce new types of adapters into the program without breaking the existing client code, as long as they work with the adapters through the client interface.

The overall complexity of the code increases because you need to introduce a set of new interfaces and classes. Sometimes it's simpler just to change the service class so that it matches the rest of your code.

Adapter : implementation approaches

- Clients call operations on an Adapter instance. In turn, the adapter calls Adaptee operations that carry out the request
- find a "narrow" interface for Adaptee, that is, the smallest subset of operations that lets us do the adaptation



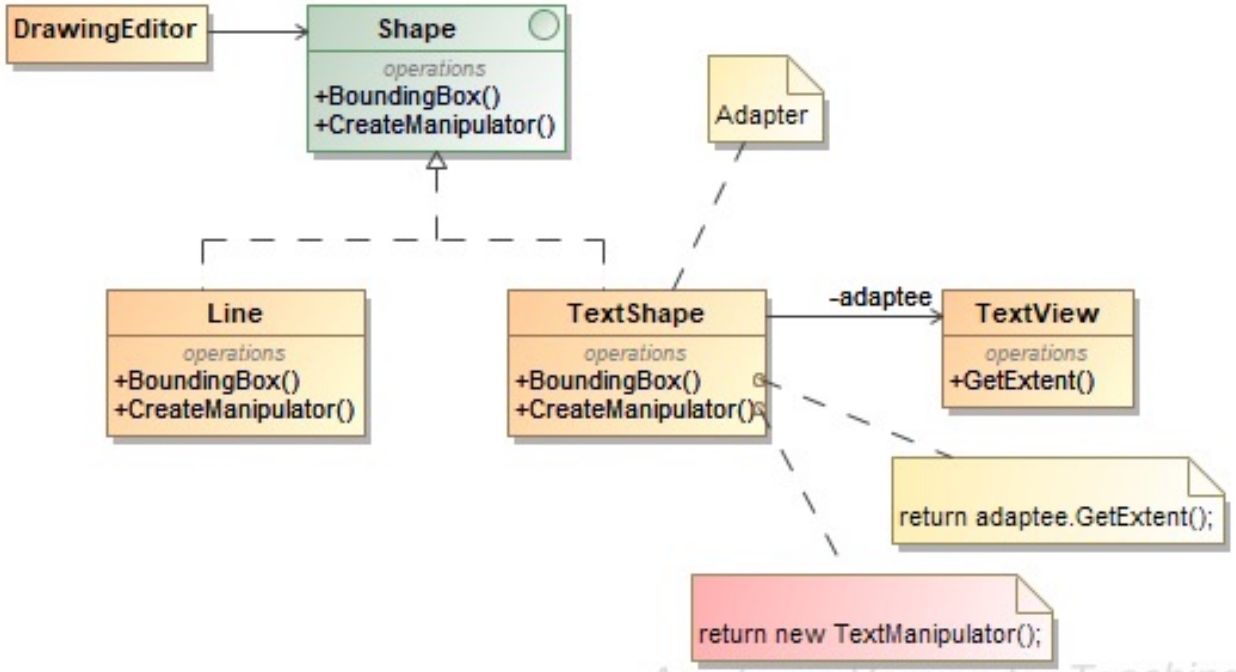
How much should the Adapter do?

- It should do everything possible to adapt



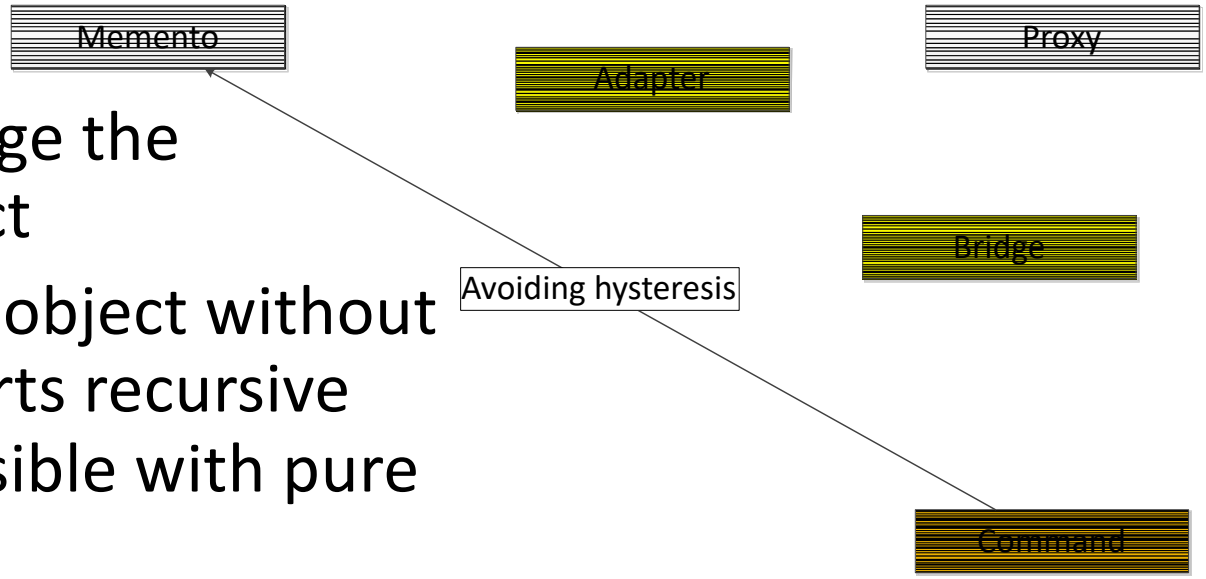
Adapter: Implementation specific

If the adapter is responsible for functionality (`CreateManipulator`) the adaptee (`TextView`) doesn't provide, then the adapter has to fulfill such responsibilities.



Academic Version for Teaching

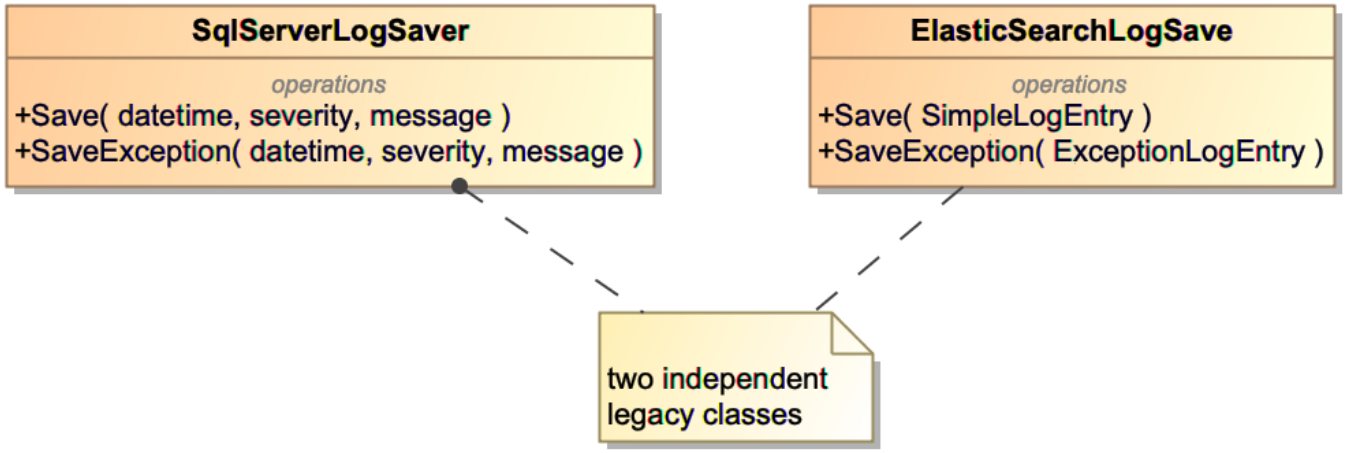
Adapter : related patterns



- An adapter is meant to change the interface of an existing object
- Decorator enhances another object without changing its interface, supports recursive composition, which isn't possible with pure adapters.
- Bridge has a similar structure but different intent: separate an interface from its implementation so that they can be varied easily and independently.
- Proxy defines a representative or surrogate for another object and does not change its interface

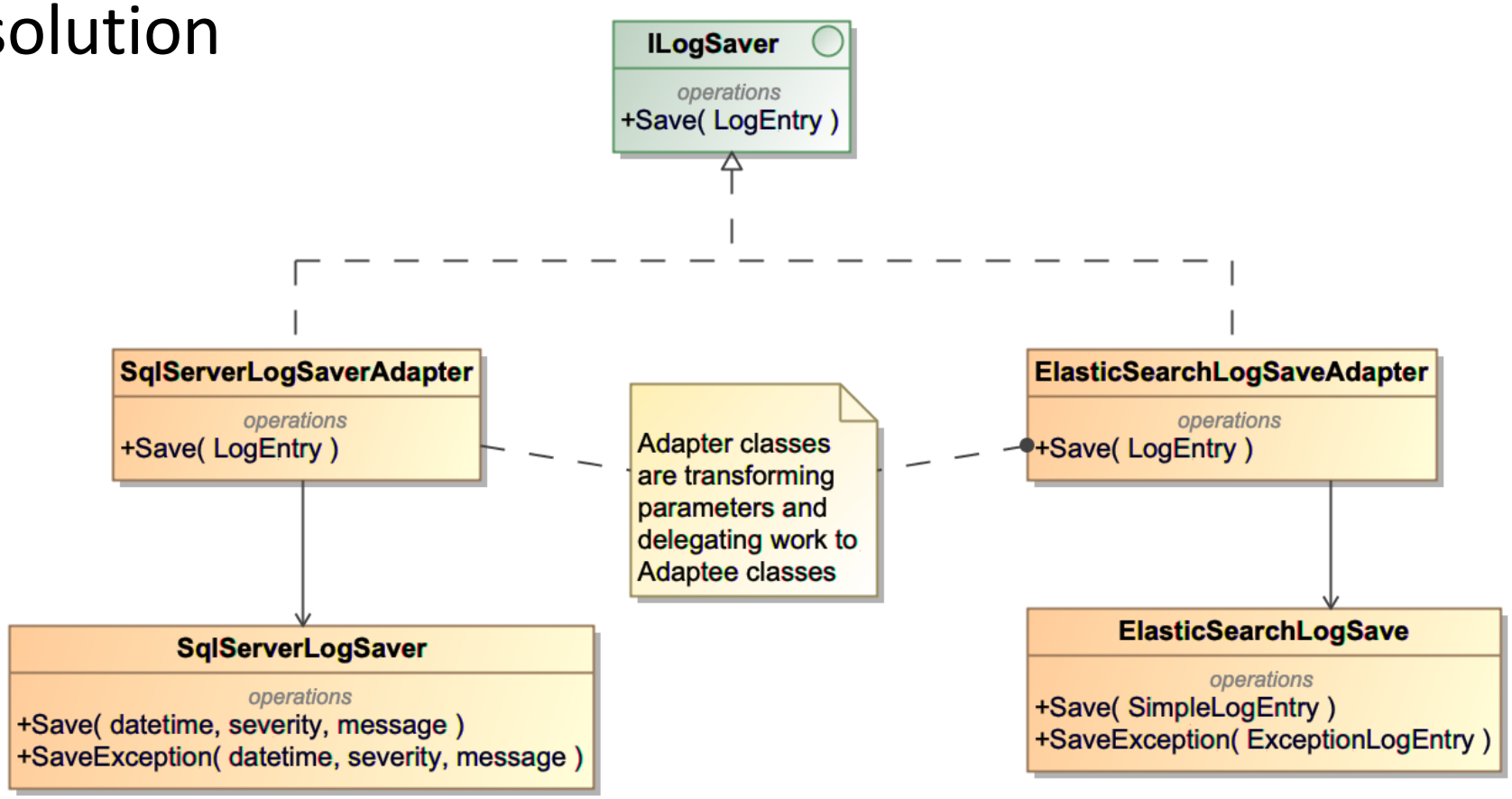
Applying adapter 1/2

- The problem: having two legacy classes, how to adopt them to a new interface?



Applying adapter 2/2

- The solution

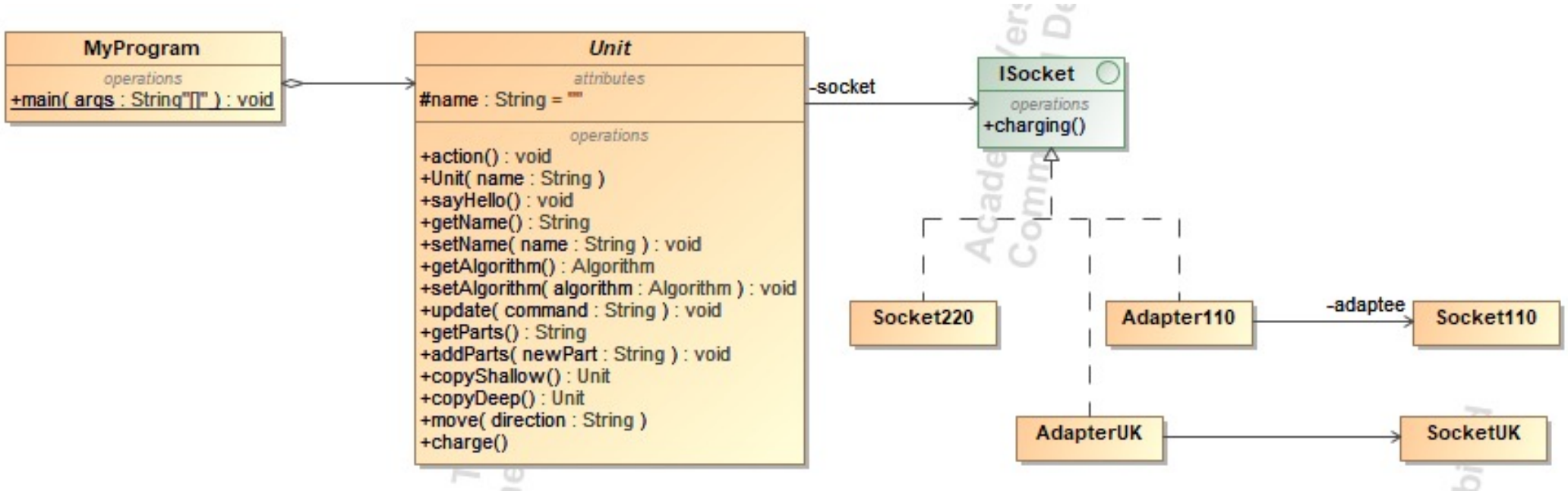


Adapter: ideas to code?

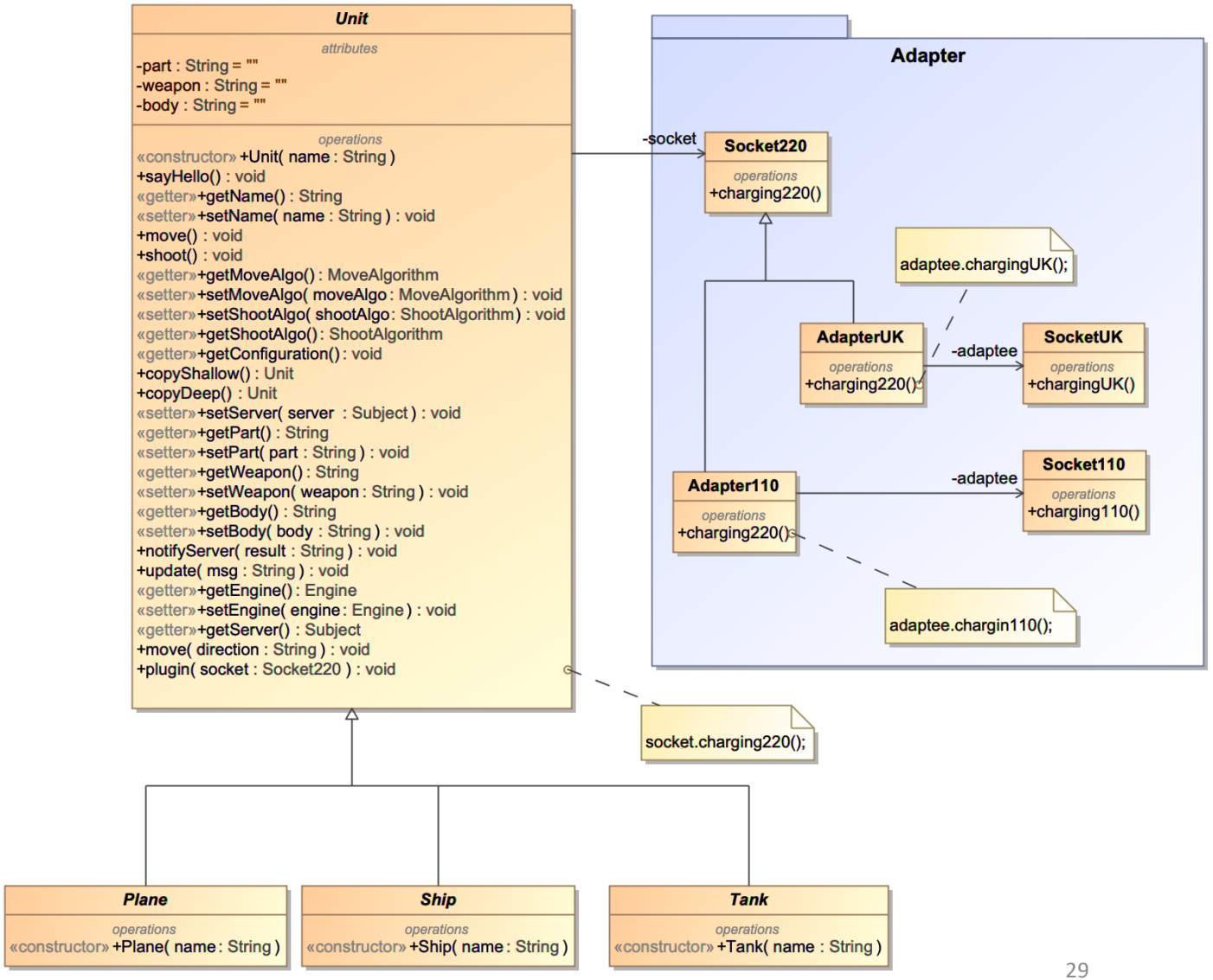
Dirrerent charging sockets for Units

- Socket220
- Socket110
- SocketUK

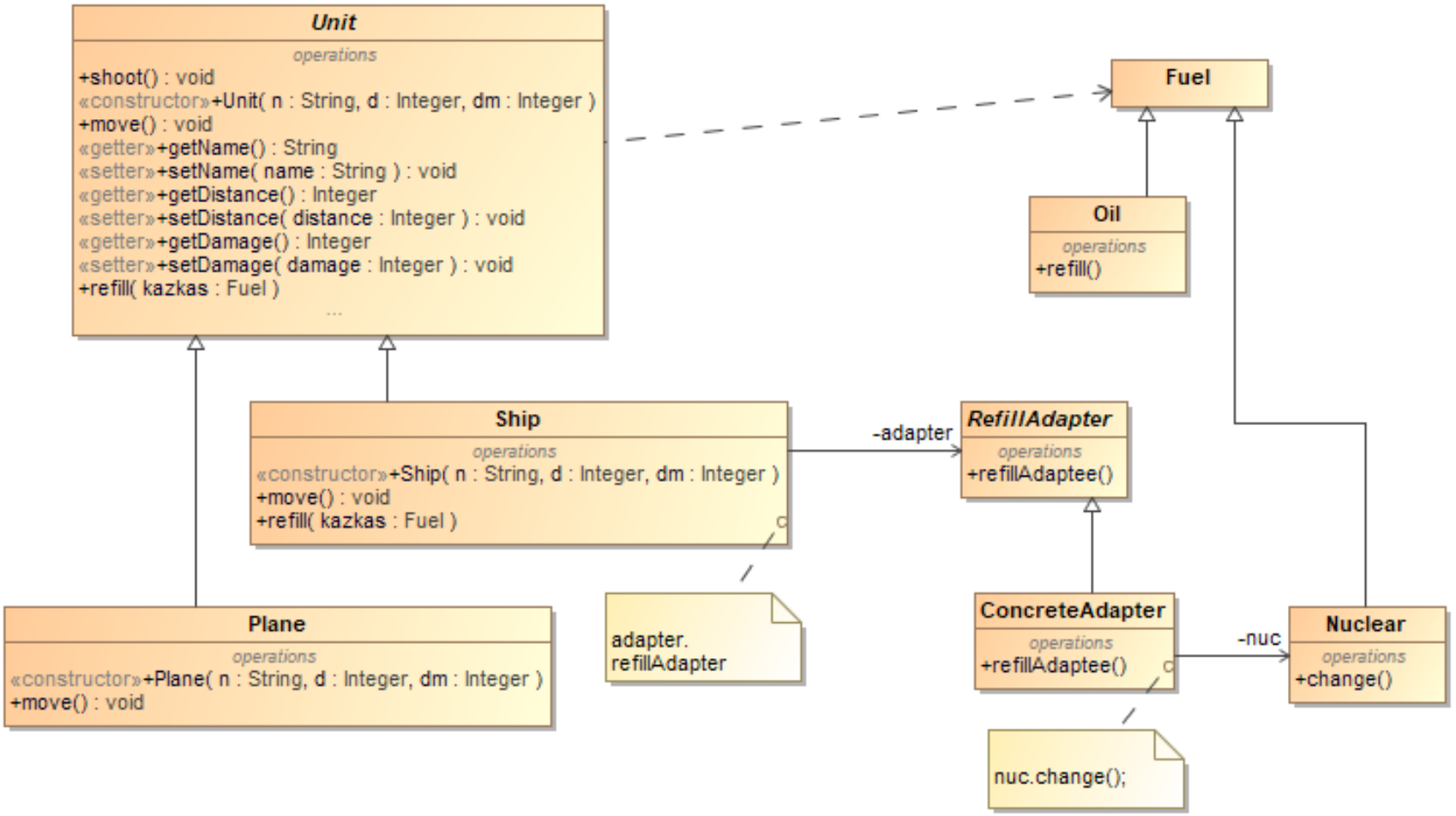
Adapter example with Sockets



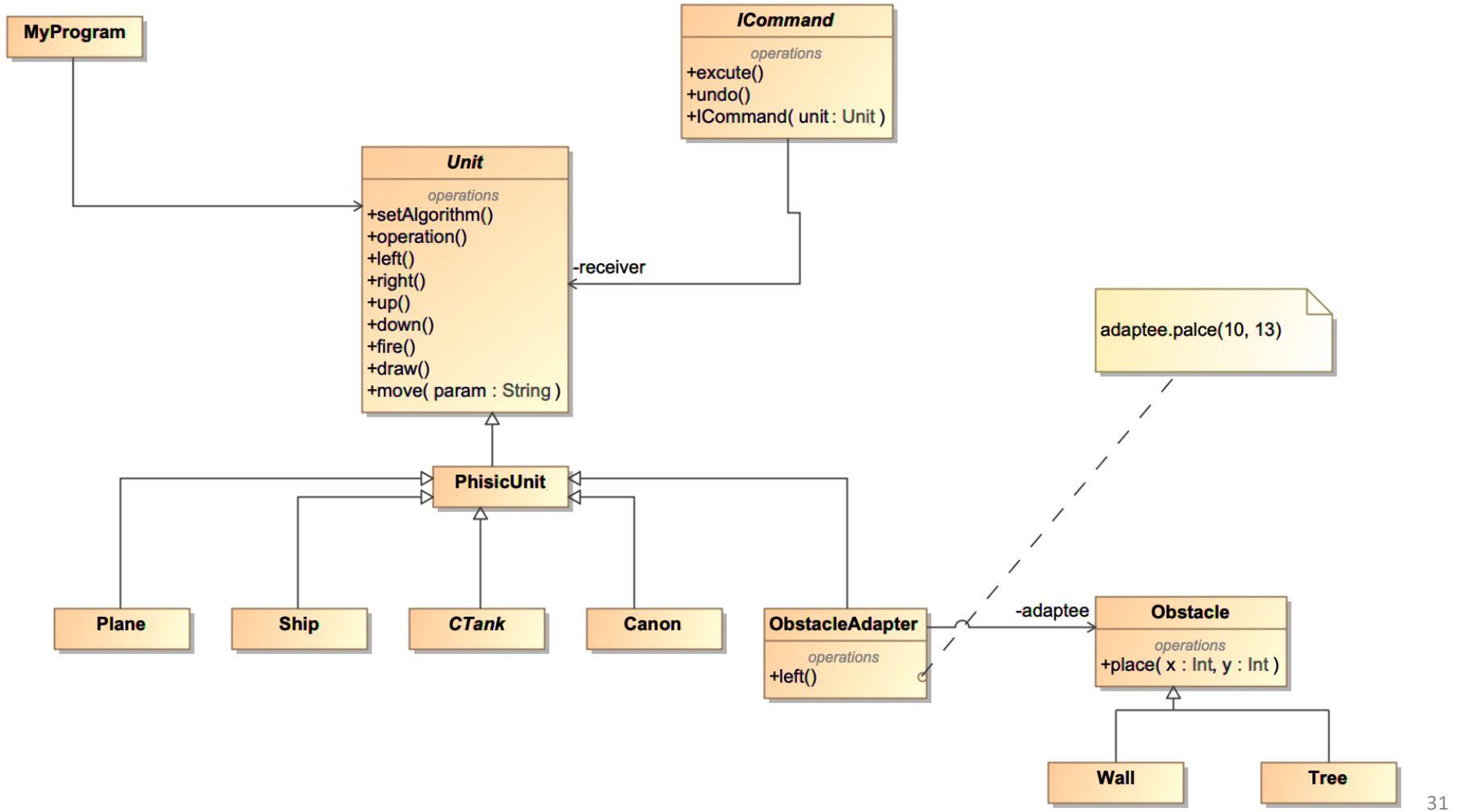
Adapter another Sockets example



Adapter Example for refueling

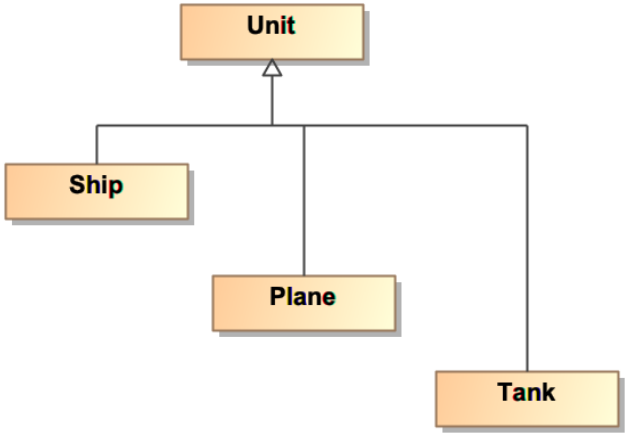


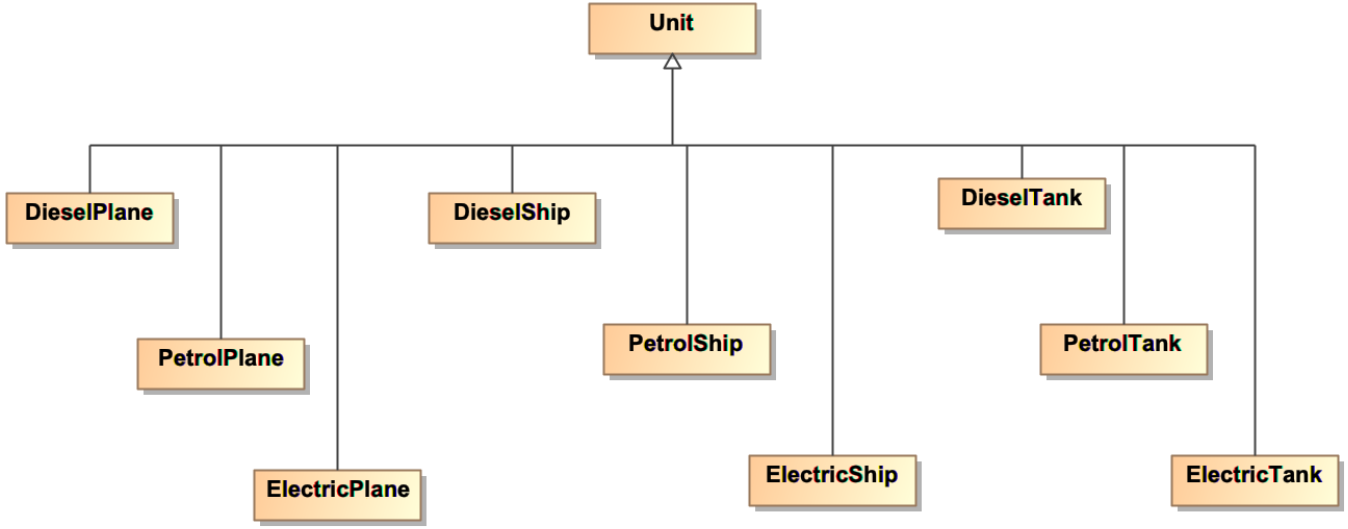
Adapter example for Obstacles



Turime unit’us bet jų varikliai keičiasi nepriklausomai

- Diesel, Petrol, Electirc





- O jei po metų atsiras Hybrid variklis? Arba Motorcycle?

Bridge - kai vienu metu nepriklausomai keičiasi dvi hierarchijos

- Type: Structural
- Also Known As: Handle/Body
- Intent: “**Decouple** an **abstraction** from its **implementation** so that the two can vary independently”

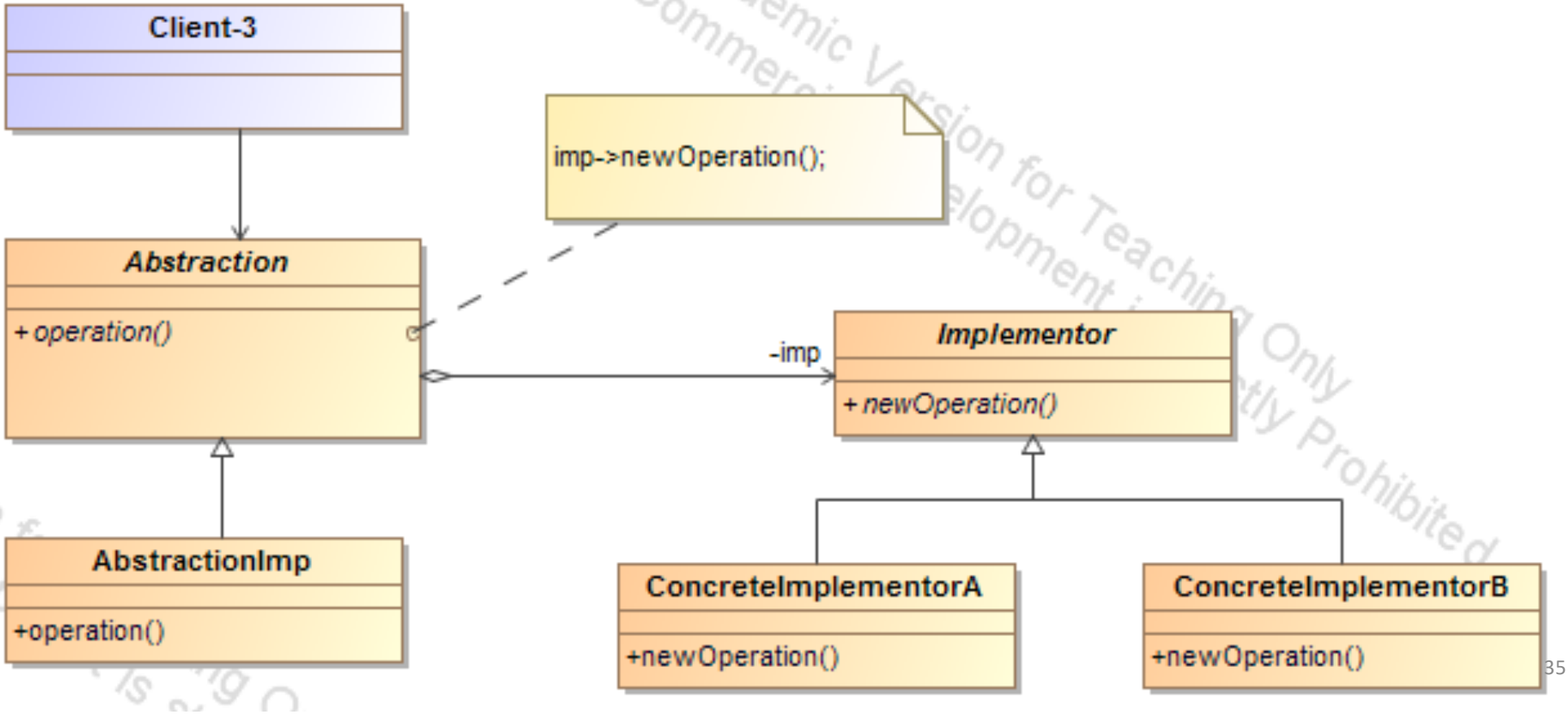
Motivation to use “Bridge”

- When an **abstraction** can have one of several possible implementations, the usual way to accommodate them is to use inheritance.

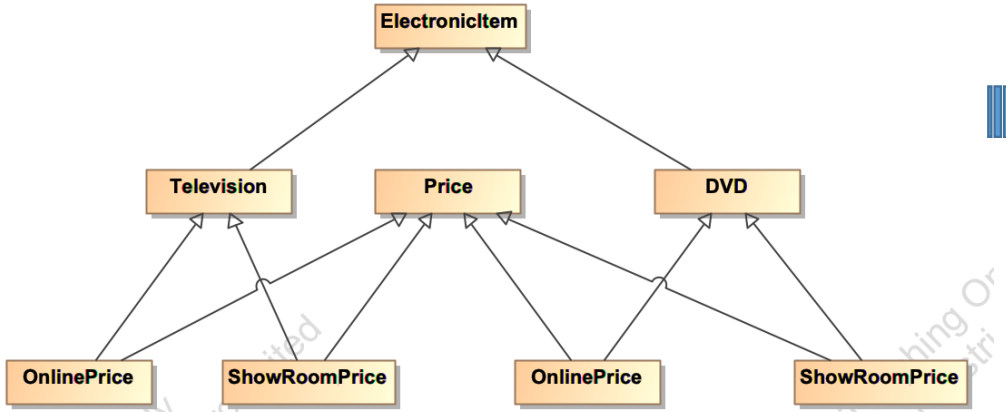
- *An abstract class defines the interface to the abstraction, and concrete subclasses implement it in different ways. But this approach isn't always flexible enough.* ***Inheritance binds an implementation to the abstraction*** *permanently, which makes it difficult to modify, extend, and reuse abstractions and implementations independently*

Kaip GoF sprendžia problemą

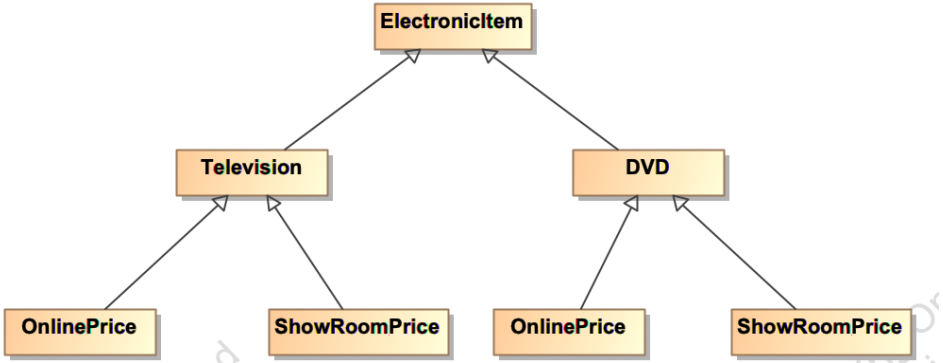
Bridge's structure



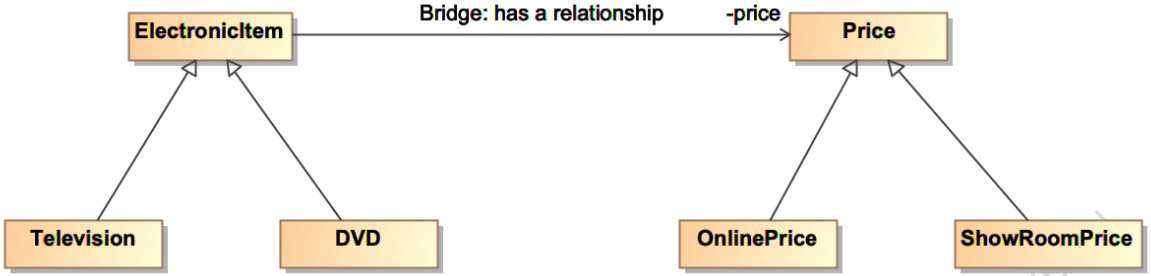
Example for Bridge problem



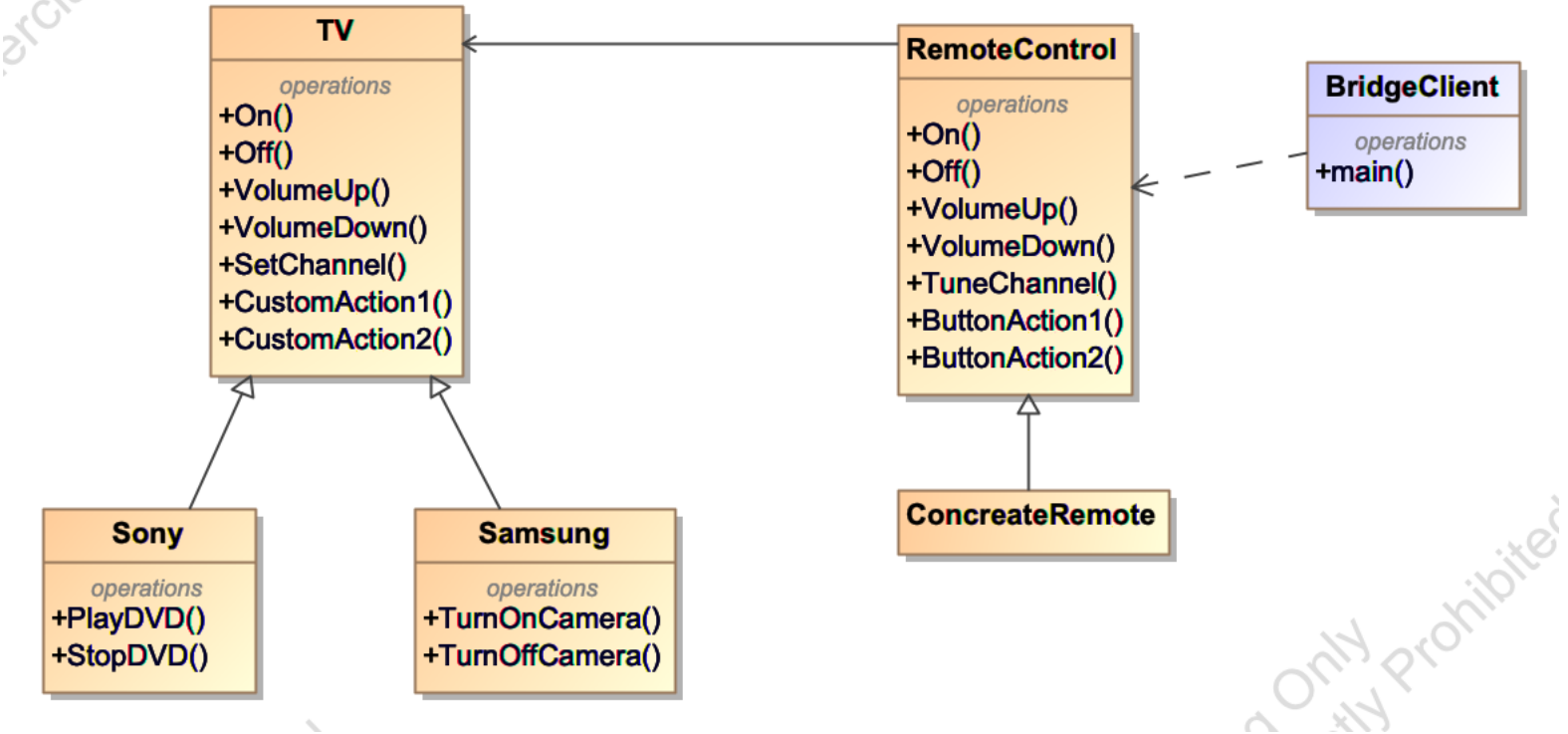
1



2



Bridge : example



Bridge Pros and Cons

You can create platform-independent classes and apps.

The client code works with high-level abstractions. It isn't exposed to the platform details.

Open/Closed Principle. You can introduce new abstractions and implementations independently from each other.

Single Responsibility Principle. You can focus on high-level logic in the abstraction and on platform details in the implementation.

You might make the code more complicated by applying the pattern to a highly cohesive class.

Bridge : design insight

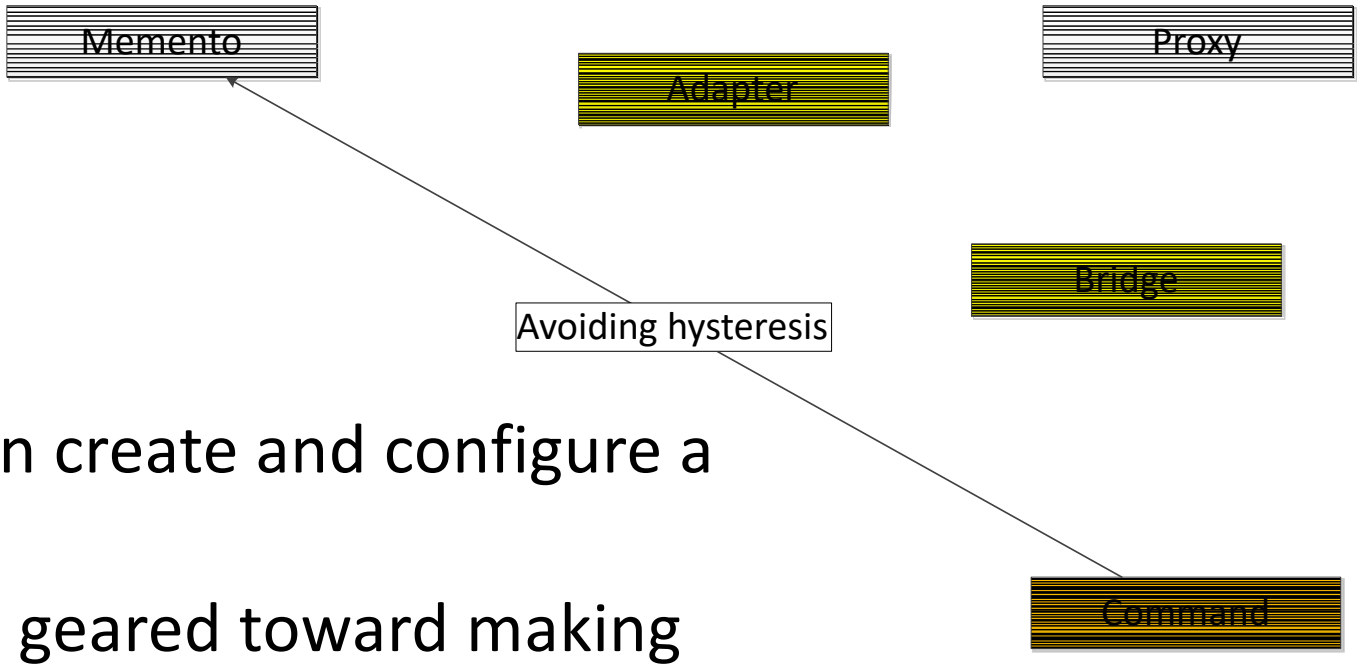
- Abstraction forwards client requests to its Implementor object.
- Decoupling interface and implementation: The implementation of **an abstraction can be configured at run-time**.
- Improved extensibility. You can **extend the Abstraction and Implementor hierarchies independently**

Bridge : implementation

How, when, and where do you decide **which Implementor class to instantiate** when there's more than one?

- Can instantiate one of them in its constructor; it can decide between them **based on parameters** passed to its constructor
- Choose a **default implementation** initially and **change it later** according to usage
- **Delegate** the decision to another object (**Factory**)

Bridge : related patterns



- An Abstract Factory can create and configure a particular Bridge.
- The Adapter pattern is geared toward making unrelated classes work together. It is usually applied to systems after they're designed.
- Bridge, on the other hand, is used up-front in a design to let abstractions and implementations vary independently.

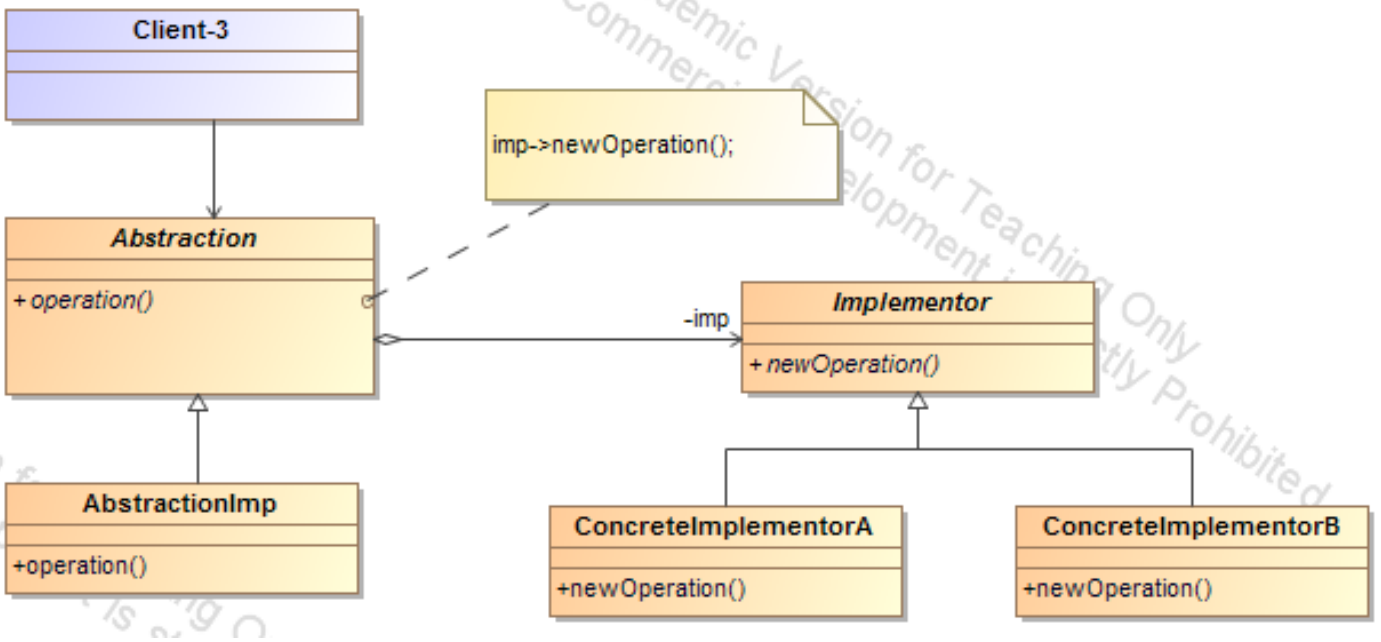
What is the difference between the bridge pattern and the … strategy pattern?

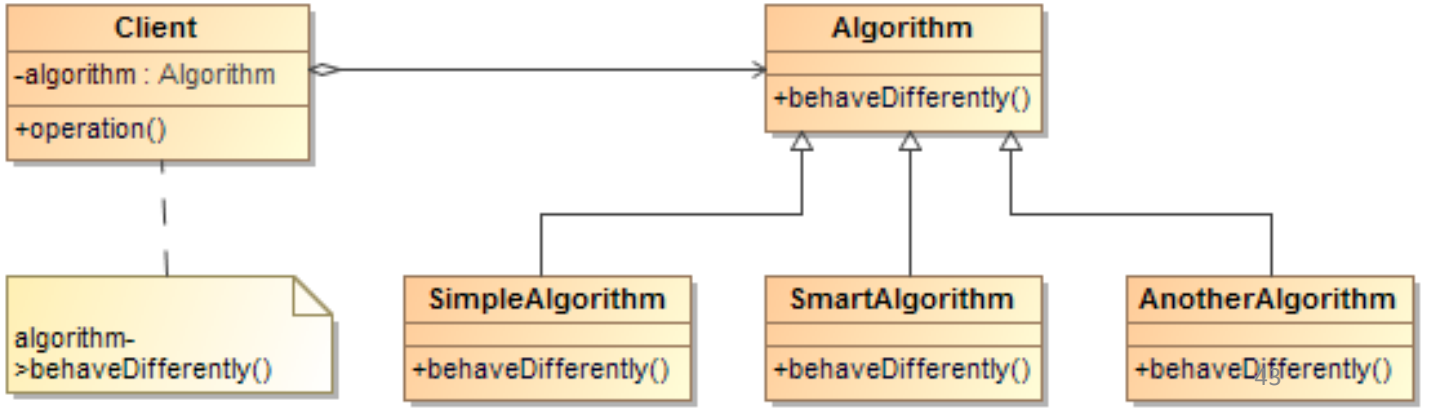
- Bridge
- *Type*: Structural
- *Intent*: “Decouple an abstraction from its implementation so that the two can vary independently”

- Strategy
- *Type*: Behavioral
- *Intent*: “Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”

Bridge vs Strategy

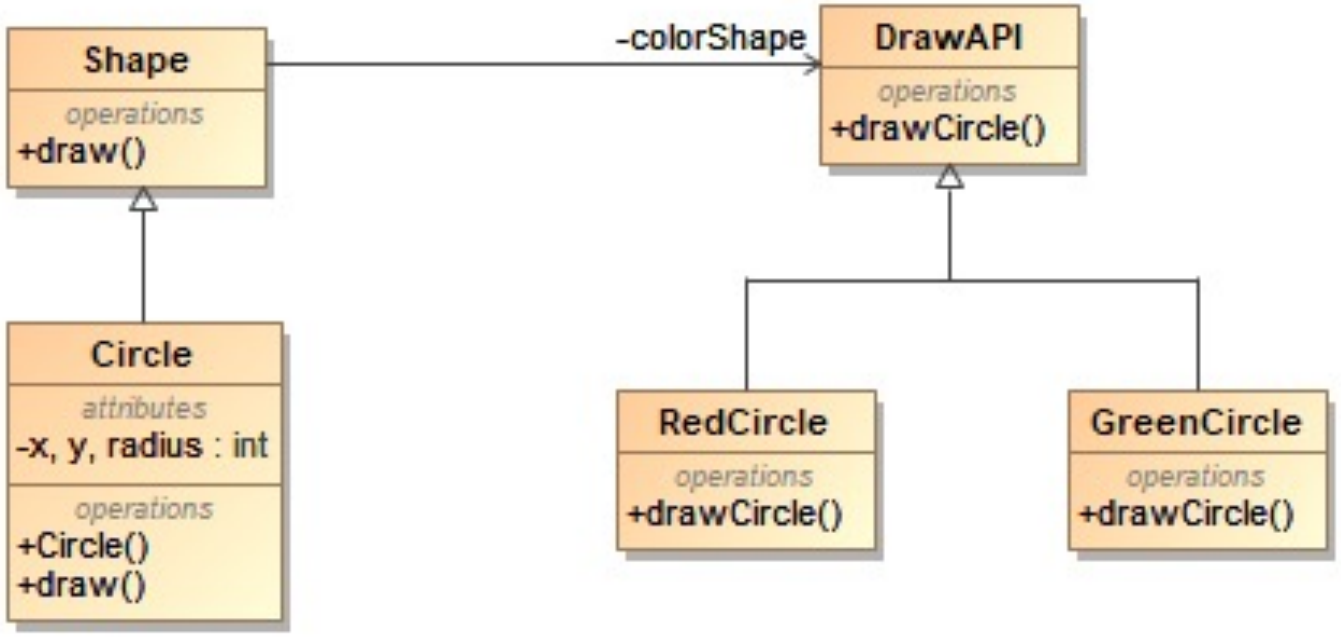
- What is part and what is whole?
- Inheritance for whole.
- Aggregation for part.





Bridge: wrong reasoning makes us get confused

Separating “Alarm Clock” implementation from its abstraction ☺



Bridge: ideas to apply

Both Abstractor and Implementor are complex objects extending their hierarchies independently:

1) Install Engine into Vehicle:

   Diesel or Petrol or Electric

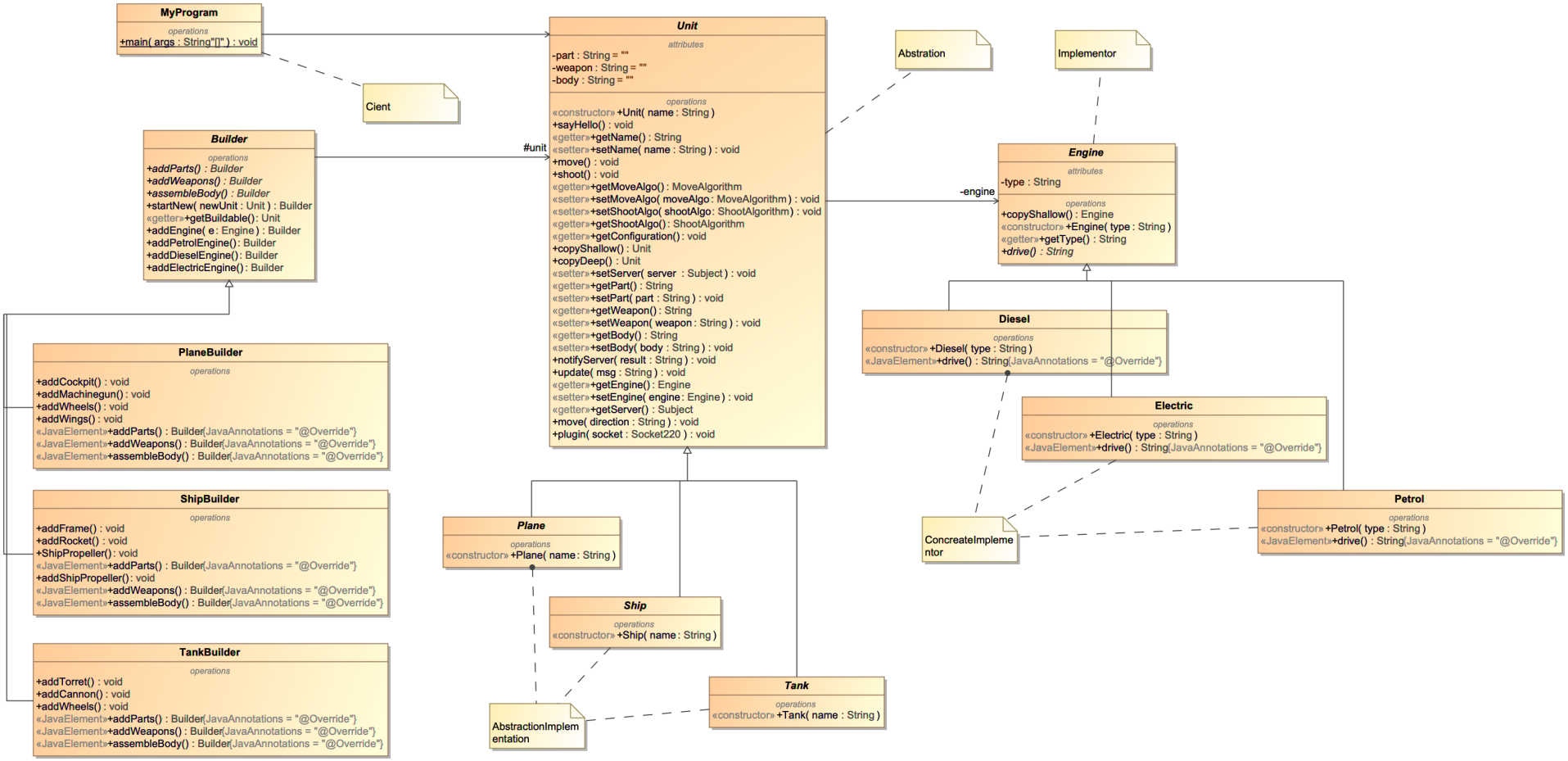
2) Install an OS into a Computer:
   - Different Computer platforms: PC, MAC, etc.
   - Different OS: Android, iOS, Windows, etc.

3) Depending on region
   - display differently Date&Time, Currency, Language formats

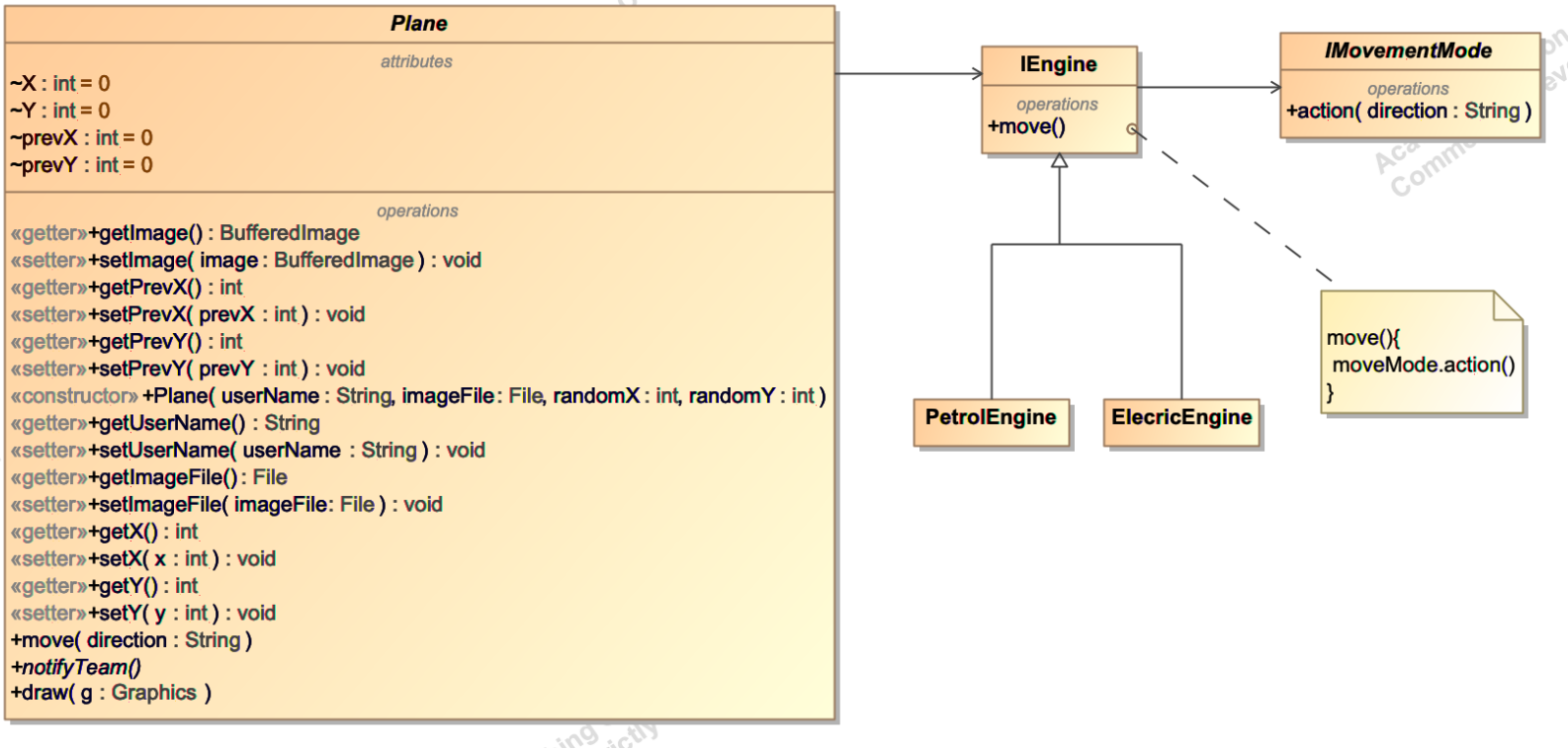
Bridge : example to code

- Use previously created Engines (as implementor) and Units (as abstraction)

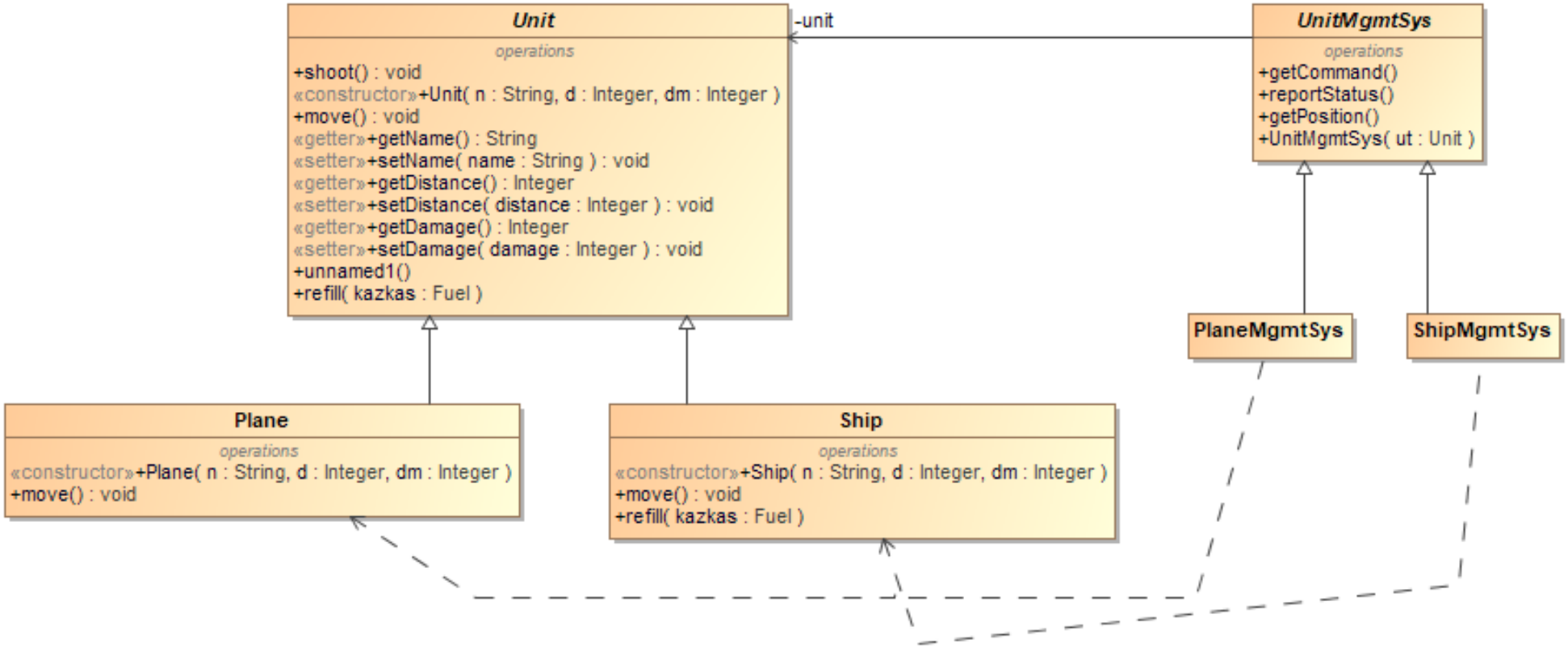
Bridge example with Engines



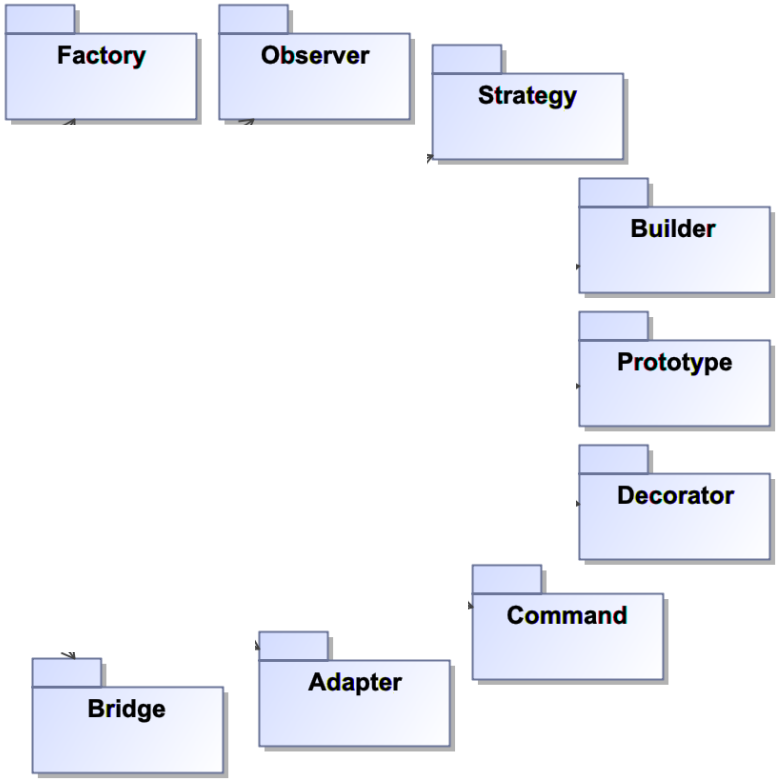
Bridge: another one Engine example in conjunction with the strategy pattern



Bridge example: Unit and its Management system



Turime žaidimą su Unit'ais... kuriems sukūrėme daug funkcionalumo, pritaikant šablonus



Façade - nesukuriant naujo funkcionalumo, paslėpia sudėtingumą už vienos paprastos sąsajos

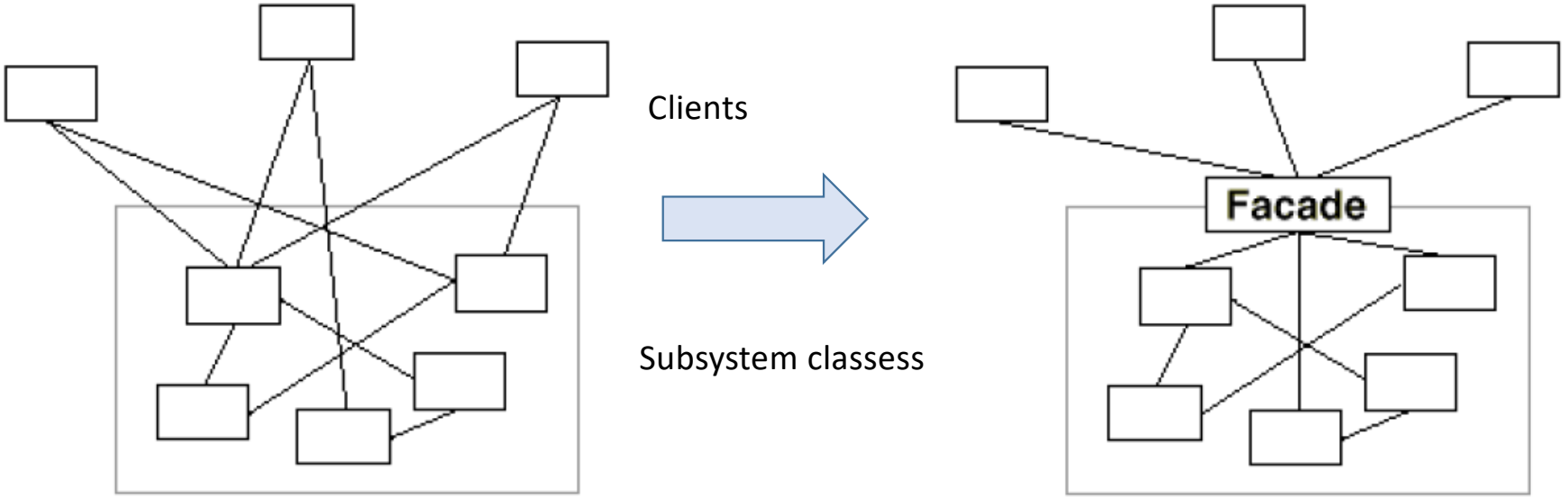
- Type: Structural

- Also Known As:

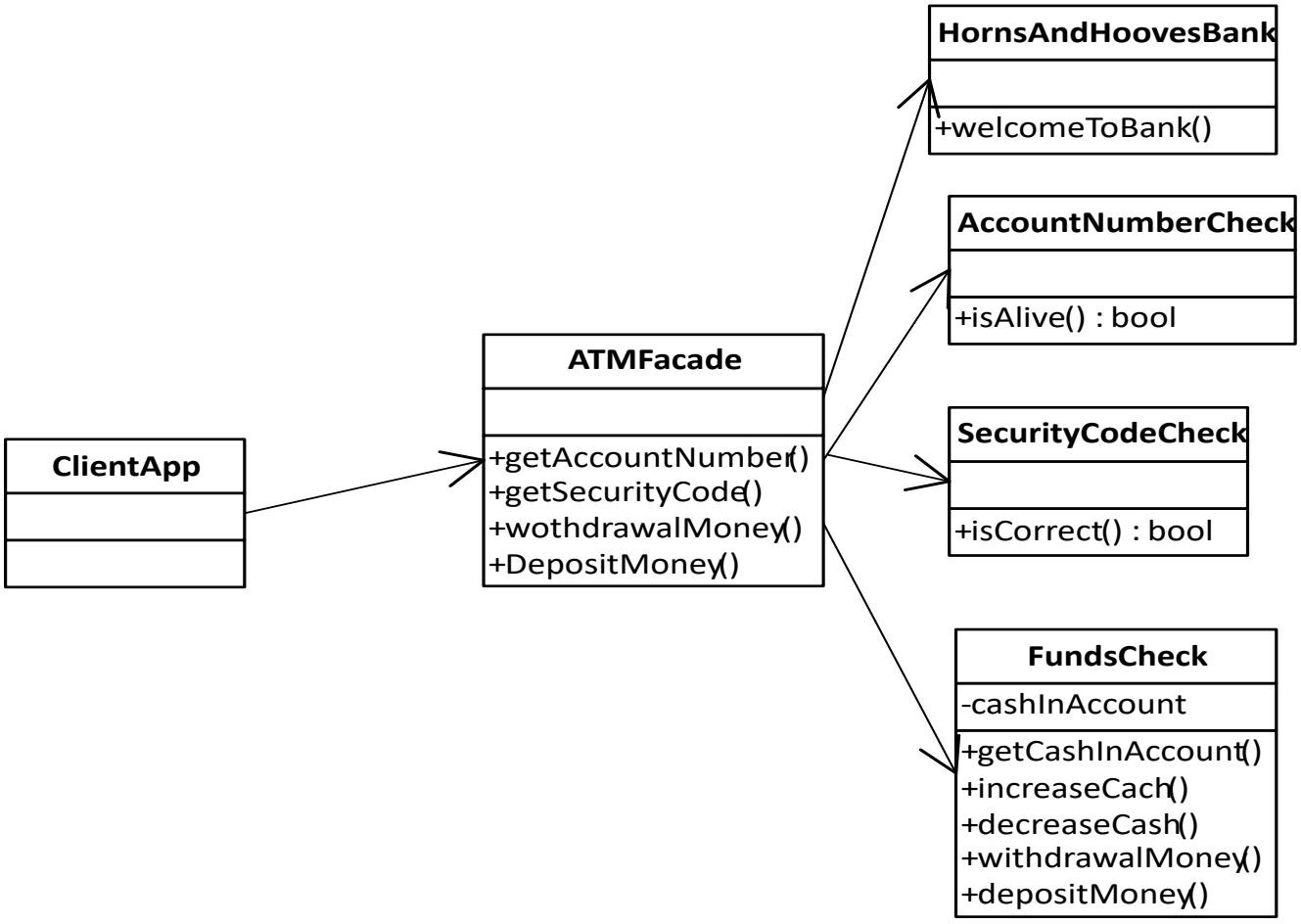
- Intent: “Provide a **unified interface** to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use”
- Simplified interface that **performs many other action behind the scenes**

When to use “Facade”

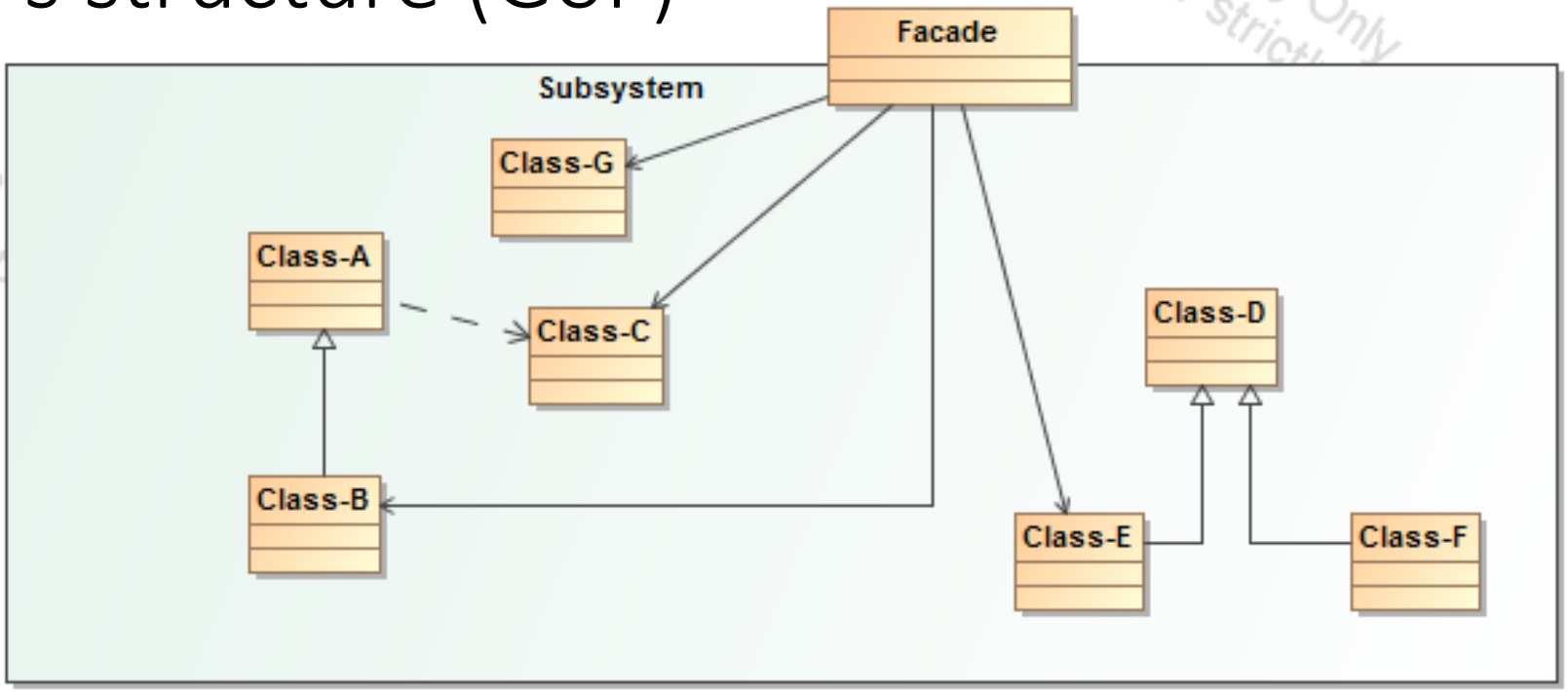
- you want to provide a simple interface to a complex subsystem
- If you need to minimize the communication and dependencies between clients and implementation classes and decouple them
- you want to layer your subsystems



Facade : the example of ATM



Facade's structure (GoF)



- knows which subsystem classes are responsible for a request.
- delegates client requests to appropriate subsystem objects
- subsystem classes keep no references to Facade

Facade Pros and Cons

You can isolate your code from the complexity of a subsystem.

A facade can become **a god object** coupled to all classes of an app.



Facade : design insight

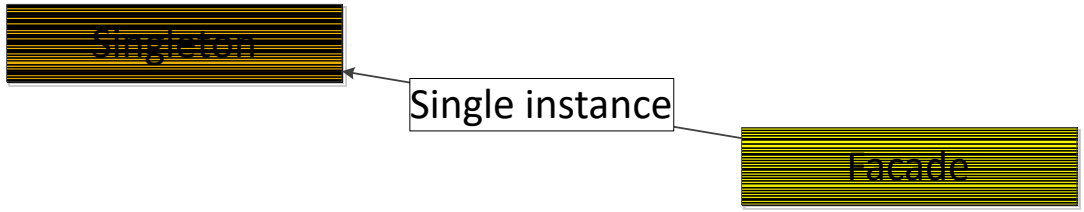
- Promotes **weak coupling** between the subsystem and its clients. Weak coupling lets you vary the components of the subsystem without affecting its clients

- It doesn't prevent applications from using subsystem classes if they need to. Thus you can choose between ease of use and generality

Facade : implementation

- The **coupling** between clients and the subsystem (third party libraries or subsystems) can be **reduced** even further by making **Facade an abstract class** with concrete subclasses for **different implementations** of a subsystem

Facade : related patterns



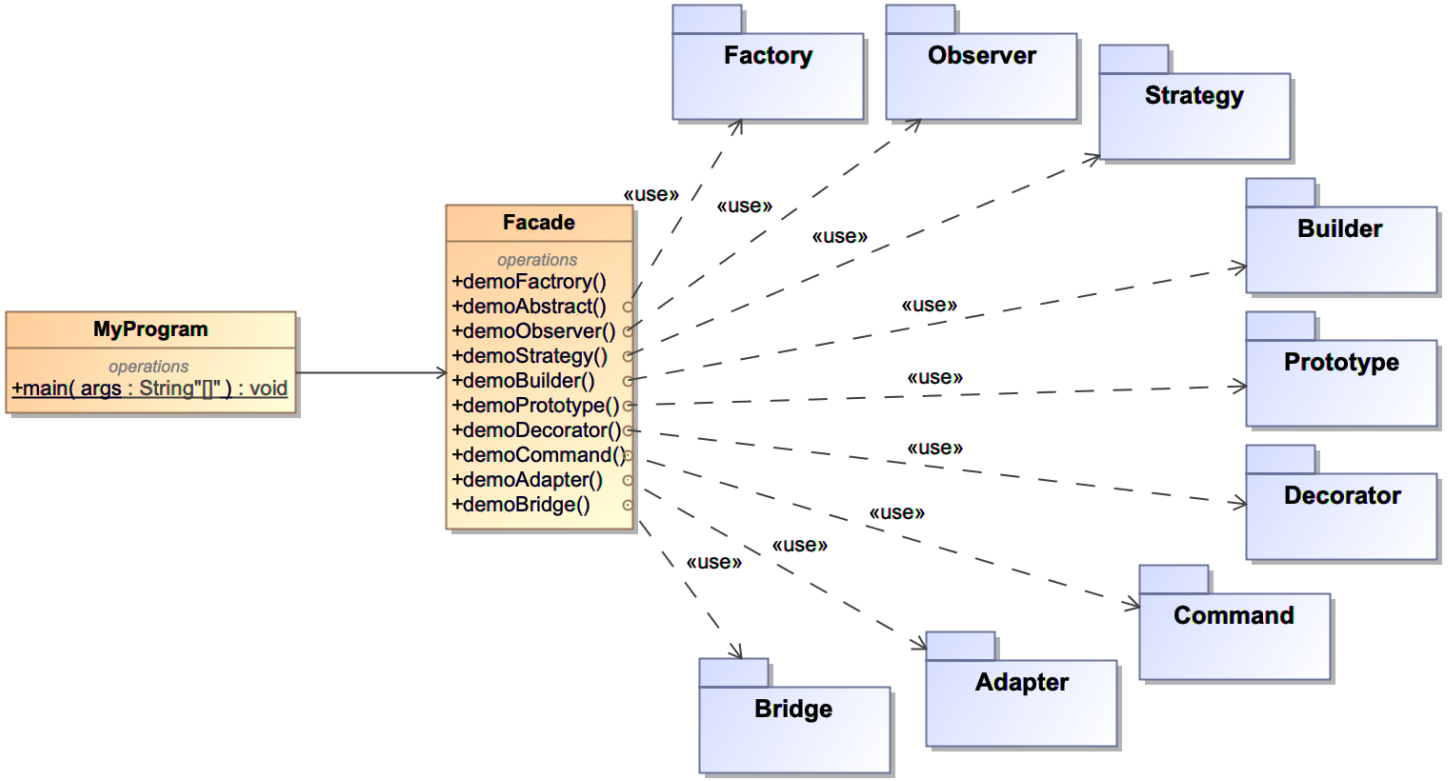
- Facade objects are often Singletons
- Abstract Factory can be used with Facade to provide an interface for creating subsystem objects in a subsystem-independent way.
- Abstract Factory can also be used as an alternative to Facade to hide platform-specific classes.
- Mediator is similar to Facade in that it abstracts functionality of existing classes. Colleagues are aware of and communicate with the mediator instead of communicating with each other directly.

Facade : example to code

- Use Facade to execute all previous examples

- Use Façade to:
  - simplify client-server communication,
  - decouple from used concrete library,
  - Simplify upgrade to new version of the library,
  - Switching to another library will require only Adapter pattern

Facade : created during lecture





Ką šiandien išmokome?

- **Adapter** - Turime dvi jau egzistuojančias sistemas ir norime jas sujungti.

- **Bridge** - Iš anksto projektuojame sistemą taip, kad dvi hierarchijos galėtų vystytis nepriklausomai

- **Façade** – slepiam sudėtingumą už paprastos sąsajos

Klausimai?

